



Basi di Dati
Progetto A.A. 2025/2026

BACHECA ELETTRONICA DI ANNUNCI

0310935

Matteo Masini

Indice

1.	Descrizione del Minimondo.....	2
2.	Analisi dei Requisiti	3
3.	Progettazione concettuale	7
4.	Progettazione logica	15
5.	Progettazione fisica	26

1. Descrizione del Minimondo

- | | |
|----|--|
| 1 | Si vuole realizzare un sistema informativo per la gestione di una bacheca elettronica di |
| 2 | annunci. Tale bacheca permette agli utenti del sistema di inserire annunci per la vendita di |
| 3 | materiale usato, di scambiare messaggi tra di loro (in maniera privata) per accordarsi sulla |
| 4 | vendita/consegna dell'oggetto, o di inserire domande (in maniera pubblica) sull'oggetto. |
| 5 | Un utente del sistema si registra scegliendo un username univoco, inserendo tutte le sue |
| 6 | informazioni anagrafiche, indicando un indirizzo di residenza ed eventualmente un indirizzo |
| 7 | di fatturazione, un numero arbitrario di recapiti (telefono, cellulare, email) indicandone uno |
| 8 | come mezzo di comunicazione preferito. |
| 9 | I gestori del servizio possono creare una gerarchia di categorie per gli annunci. Un utente, |
| 10 | per creare un annuncio, seleziona una categoria e scrive una descrizione dell'oggetto. |
| 11 | Quando un oggetto inserito in bacheca è stato venduto, l'utente lo indica come tale e questo |
| 12 | non viene più visualizzato nella bacheca pubblica. |
| 13 | Un utente del sistema, una volta letto e scelto un annuncio, può decidere di inserire un |
| 14 | commento pubblico o di inviare un messaggio privato all'utente che ha inserito l'annuncio. |
| 15 | Similmente, un utente può "seguire" uno degli annunci, venendo così informato ogni volta |
| 16 | che su questo viene effettuata una modifica (ad esempio, viene inserita una nuova nota). |
| 17 | I gestori del servizio possono generare un report indicante in forma aggregata per ciascun |
| 18 | utente del sistema quale percentuale di annunci pubblicati sono stati contrassegnati come |
| 19 | venduti. |

2. Analisi dei Requisiti

Identificazione dei termini ambigui e correzioni possibili

Linea	Termine	Nuovo termine	Motivo correzione
1	Sistema informativo	Sistema informatico	Un sistema informativo è l'insieme generale di tutte le risorse, informazioni, persone e procedure organizzative necessarie per gestire i dati. Adotto il termine sistema informatico per indicare nello specifico l'infrastruttura tecnologica e l'applicazione software con cui gli utenti interagiscono per inserire e consultare gli annunci.
4, 10, 11	Oggetto	Bene	Il termine oggetto è generico e ambiguo in informatica. Bene definisce con precisione la risorsa fisica reale messa in vendita nell'annuncio.
4	Domande(in maniera pubblica)	Commenti	Termine più preciso per un messaggio pubblicato su un annuncio.
6	Tutte informazioni anagrafiche	Dati anagrafici	Termine per definire con esattezza quali informazioni personali sono necessarie per identificare l'utente al momento della registrazione, evitando la richiesta di dati personali superflui o non permanenti.
7	Recapiti	Contatti	Il termine recapiti risulta d'uso comune ma generico. Utilizzo il termine contatti per definire in modo diretto ed essenziale l'insieme dei vari riferimenti specificati dall'utente.
8	Mezzo di comunicazione	Metodo di contatto	Il termine mezzo di comunicazione risulta troppo formale e generica. Con il termine metodo di contatto ci si collega direttamente ai contatti inseriti dall'utente.
9, 17	Gestori del servizio	Amministratori del sistema	Distinzione del ruolo con privilegi operativi dal normale utente del sistema.
15	Seguire	Monitorare	Termine più adatto al contesto di una base di dati.

Specifica disambiguata

Si vuole realizzare un **sistema informatico** per la gestione di una bacheca elettronica di annunci. Tale bacheca permette agli utenti del sistema di inserire annunci per la vendita di materiale usato, di scambiare messaggi tra di loro (in maniera privata) per accordarsi sulla vendita/consegna del **bene**, o di inserire **commenti** sul **bene**.

Un utente del sistema si registra scegliendo un username univoco, inserendo i suoi **dati anagrafici**, indicando un indirizzo di residenza ed eventualmente un indirizzo di fatturazione, un numero

arbitrario di **contatti** (telefono, cellulare, email) indicandone uno come **metodo di contatto** preferito. **Gli amministratori del sistema** possono creare una gerarchia di categorie per gli annunci. Un utente, per creare un annuncio, seleziona una categoria e scrive una descrizione del **bene**. Quando un **bene** inserito in bacheca è stato venduto, l'utente lo indica come tale e questo non viene più visualizzato nella bacheca pubblica.

Un utente del sistema, una volta letto e scelto un annuncio, può decidere di avviare un'interazione in forma di commento pubblico o di messaggio privato all'utente che ha inserito l'annuncio. Similmente, un utente può **monitorare** uno degli annunci, venendo così informato ogni volta che su questo viene effettuata una modifica (ad esempio, viene inserita una nuova nota).

Gli amministratori del sistema possono generare un report indicante in forma aggregata per ciascun utente del sistema quale percentuale di annunci pubblicati sono stati contrassegnati come venduti.

Glossario dei Termini

Termine	Descrizione	Sinonimi	Collegamenti
Utente	Utente che può pubblicare annunci per vendere materiale usato, scambiare messaggi privati, inserire commenti sui beni e monitorare annunci di interesse per ricevere aggiornamenti.	Inserzionista, Utente registrato	Contatto, Annuncio, Messaggio
Contatto	Singolo riferimento fornito dall'utente per la reperibilità.	Recapito, Riferimento	Utente
Amministratore	Utente con funzioni di gestione che si occupa di creare e strutturare la gerarchia delle categorie della bacheca e di generare il report aggregato sulle percentuali di vendita degli annunci per ciascun utente.	Gestore	Categoria, Utente, Report, Annuncio
Categoria	Sezione tematica inserita in una gerarchia creata dagli amministratori per organizzare e	Tipologia, Sezione	Amministratore, Annuncio

	classificare gli annunci all'interno della bacheca elettronica.		
Interazione	Azione comunicativa svolta da un utente in relazione a un annuncio.	Comunicazione, Commento, Messaggio	Utente, Annuncio
Annuncio	Inserzione pubblicata da un utente all'interno di una categoria con la descrizione del bene posto in vendita.	Inserzione	Categoria, Utente, Messaggio

Raggruppamento dei requisiti in insiemi omogenei

Frasi relative a Utente

Tale bacheca permette agli **utenti** del sistema di inserire annunci per la vendita di materiale usato, di scambiare messaggi tra di loro (in maniera privata) per accordarsi sulla vendita/consegna del bene, o di inserire commenti sul bene. Un **utente** del sistema si registra scegliendo un username univoco, inserendo i suoi dati anagrafici, indicando un indirizzo di residenza ed eventualmente un indirizzo di fatturazione, un numero arbitrario di contatti (telefono, cellulare, email) indicandone uno come metodo di contatto preferito.

[...]

Un **utente**, per creare un annuncio, seleziona una categoria e scrive una descrizione del bene. Quando un bene inserito in bacheca è stato venduto, l'**utente** lo indica come tale e questo non viene più visualizzato nella bacheca pubblica.

Un **utente** del sistema, una volta letto e scelto un annuncio, può decidere di avviare un'interazione in forma di commento pubblico o di messaggio privato all'utente che ha inserito l'annuncio. Similmente, un **utente** può monitorare uno degli annunci, venendo così informato ogni volta che su questo viene effettuata una modifica (ad esempio, viene inserita una nuova nota).

Frasi relative a Contatto

Un utente del sistema si registra scegliendo un username univoco, inserendo i suoi dati anagrafici, indicando un indirizzo di residenza ed eventualmente un indirizzo di fatturazione, un numero arbitrario di **contatti** (telefono, cellulare, email) indicandone uno come metodo di **contatto** preferito.

Frasi relative a Amministratore

Gli **amministratori** del sistema possono creare una gerarchia di categorie per gli annunci.

[...]

Gli **amministratori** del sistema possono generare un report indicante in forma aggregata per ciascun utente del sistema quale percentuale di annunci pubblicati sono stati contrassegnati come venduti.

Frasi relative a Categoria

Gli amministratori del sistema possono creare una gerarchia di **categorie** per gli annunci. Un utente, per creare un annuncio, seleziona una **categoria** e scrive una descrizione del bene.

Frase relative a Interazione

Un utente del sistema, una volta letto e scelto un annuncio, può decidere di avviare un'interazione in forma di commento pubblico o di messaggio privato all'utente che ha inserito l'annuncio.

Frase relative a Annuncio

Si vuole realizzare un sistema informatico per la gestione di una bacheca elettronica di **annunci**. Tale bacheca permette agli utenti del sistema di inserire **annunci** per la vendita di materiale usato, di scambiare messaggi tra di loro (in maniera privata) per accordarsi sulla vendita/consegna del bene, o di inserire commenti sul bene.

[...]

Gli amministratori del sistema possono creare una gerarchia di categorie per gli **annunci**. Un utente, per creare un **annuncio**, seleziona una categoria e scrive una descrizione del bene.

[...]

Un utente del sistema, una volta letto e scelto un **annuncio**, può decidere di avviare un'interazione in forma di commento pubblico o di messaggio privato all'utente che ha inserito l'**annuncio**. Similmente, un utente può monitorare uno degli **annunci**, venendo così informato ogni volta che su questo viene effettuata una modifica (ad esempio, viene inserita una nuova nota). Gli amministratori del sistema possono generare un report indicante in forma aggregata per ciascun utente del sistema quale percentuale di **annunci** pubblicati sono stati contrassegnati come venduti.

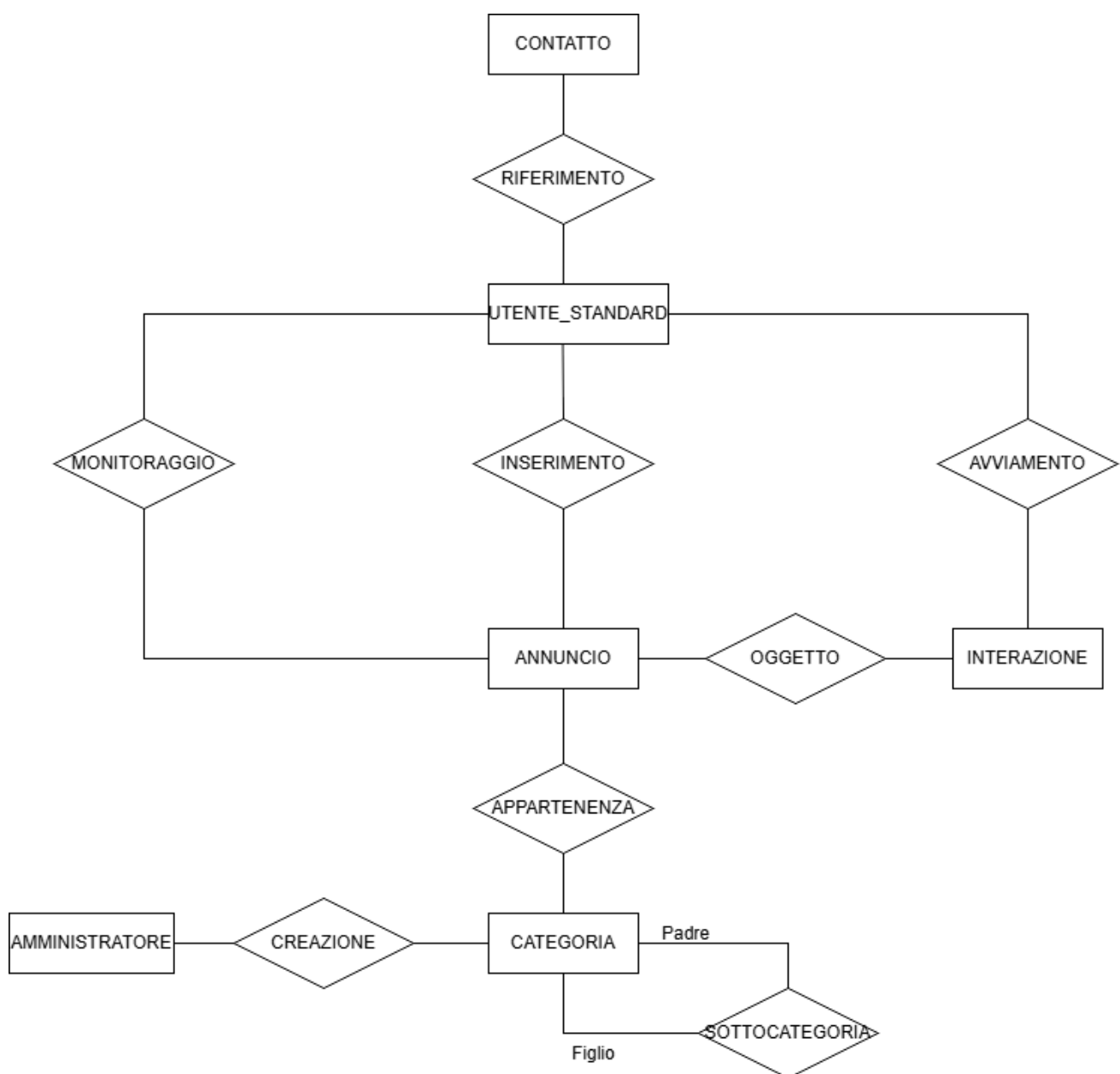
3. Progettazione concettuale

Costruzione dello schema E-R

Per la costruzione dello schema E-R ho adottato una strategia top-down.

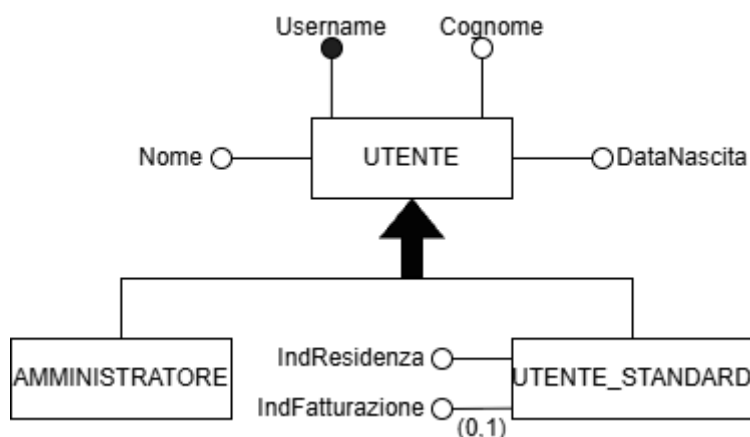
Definizione di uno schema scheletro.

Leggendo le specifiche del testo sono riuscito ad individuare le entità principali che sono “Utente_Standard”, “Contatto”, “Amministratore”, “Interazione”, “Annuncio” e “Categoria” che sono collegate tra di loro dalle relazioni che vengono mostrate nella figura di sotto:



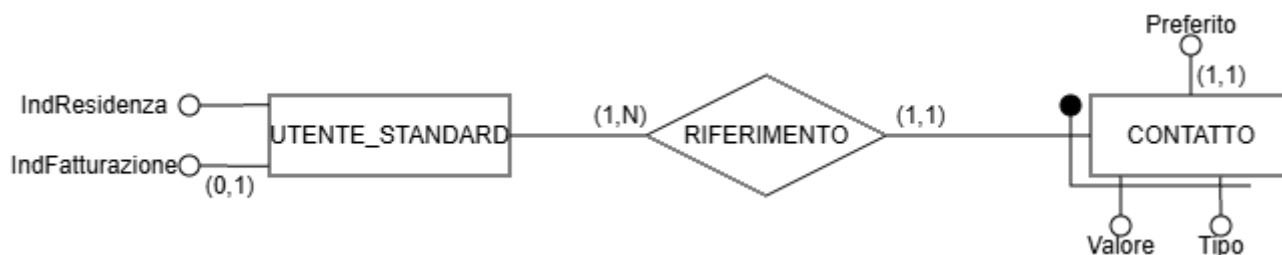
Raffinamento dell'entità “Utente”:

Sono presenti due tipi di utente, ossia “Utente_Standard” e “Amministratore”. Per rappresentare questa entità utilizzo una generalizzazione di tipo totale ed esclusiva. Di ogni “Utente_Standard” ci interessa il nome, cognome, data di nascita, username, indirizzo di residenza e indirizzo di fatturazione con cardinalità (0,1) in quanto è un'informazione opzionale e monovalore. Di ogni “Amministratore” ci interessa il nome, cognome, data di nascita e username. Attributi come nome, cognome, data di nascita e username sono comuni nelle due entità figlie e quindi li vado a mettere direttamente nell'entità genitore “Utente”. L'identificazione univoca di ciascun utente sarà data dal campo username, che è stato scelto come identificatore interno.



Inserimento e cardinalità della relazione “Riferimento” e raffinamento dell'entità “Contatto”:

Un utente standard può avere diversi metodi di contatto, di cui uno da indicare come preferito. L'entità “Contatto” è definita dagli attributi: tipo di recapito, valore del recapito e un booleano Preferito con cardinalità (1,1) che indica se è il contatto preferito dell'utente standard. L'identificatore è composto dallo username di “Utente_Standard” unito a valore e tipo di contatto. Questo legame è rappresentato con una relazione uno a molti, in cui Utente Standard partecipa con cardinalità (1,N) perché l'utente può avere da uno a molti contatti e Contatto con cardinalità (1,1) perché il recapito appartiene a uno e un solo utente.



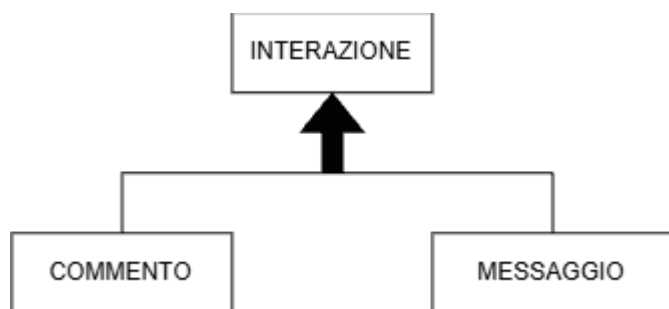
Inserimento e cardinalità della relazione “Avviamento” e raffinamento dell’entità “Interazione”:

La relazione “Avviamento” collega le entità “Utente_Standard” e “Interazione”. L’utente standard partecipa con cardinalità (0,N) poiché un utente può avviare zero o più interazioni, mentre Interazione partecipa con cardinalità (1,1) perché ogni interazione è avviata da un solo utente. L’entità “Interazione” ha come attributi DataOra che indica il timestamp e il testo. L’identificatore è lo username di utente standard collegato a DataOra.



Generalizzazione dell’entità “Interazione”:

Sono presenti due tipi di interazione, ovvero “Messaggio” e “Commento”. Per rappresentare questa entità utilizzo una generalizzazione di tipo totale ed esclusiva. Gli attributi DataOra e Testo sono comuni a tutte e due le entità figlie, quindi li ho messi direttamente nell’entità “Interazione”. Nello scheletro base non ho inserito la relazione “Ricezione”, in quanto l’ho aggiunta successivamente, nel momento in cui ho definito l’entità figlia “Messaggio”.



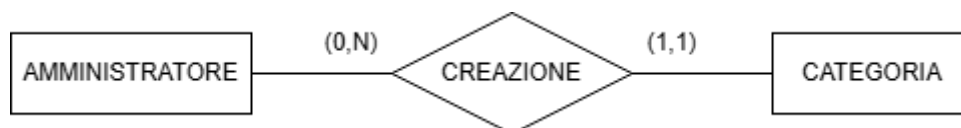
Inserimento e cardinalità della relazione “Ricezione”:

La relazione “Ricezione” descrive lo scambio di messaggi privati tra due utenti e lega l’entità “Utente_Standard” a “Messaggio”. La cardinalità di “Messaggio” è (1,1) poiché ogni singolo messaggio privato è indirizzato a uno e un solo utente specifico. La cardinalità di “Utente_Standard” è (0,N) perché un utente può non aver ancora ricevuto alcun messaggio privato nella propria casella oppure può riceverne molti nel tempo da parte di diversi utenti.



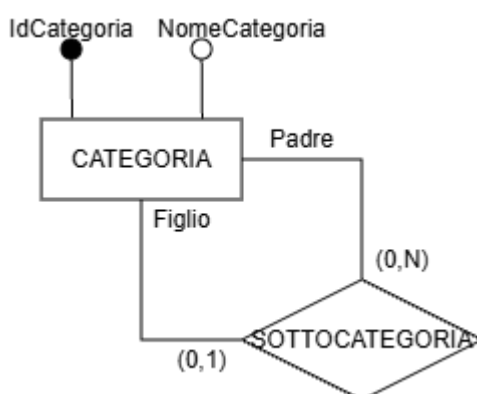
Inserimento e cardinalità della relazione “Creazione”:

La relazione “Creazione” collega le due entità “Amministratore” e “Categoria”. Amministratore partecipa con cardinalità (0,N) poiché un amministratore può creare zero o molte categorie, mentre Categoria partecipa con cardinalità (1,1) poiché una categoria è creata da un solo amministratore.



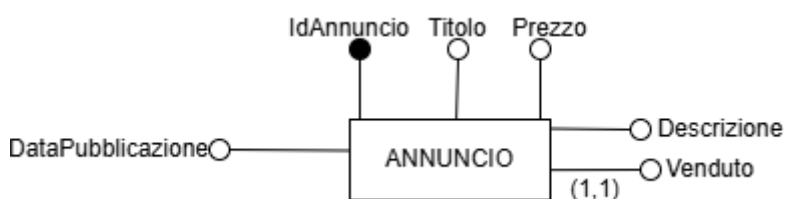
Raffinamento dell’entità “Categoria”:

Gli attributi associati all’entità “Categoria” sono: codice identificativo, il nome e l’eventuale categoria padre di cui è figlia. Per questo motivo ho aggiunto la relazione “Sottocategoria”, poiché nella specifica del testo si parla di una gerarchia di categorie. Poiché parliamo di gerarchia di categorie, ho deciso di rappresentarlo attraverso una relazione ricorsiva dove partecipa la “Categoria Padre” con cardinalità (0,N) perché può contenere da 0 a molte sottocategorie figlie e la “Categoria Figlia” con cardinalità (0,1).



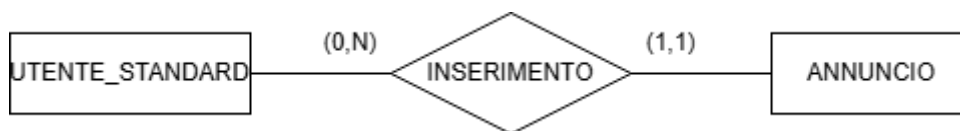
Raffinamento dell’entità “Annuncio”:

L’entità “Annuncio” rappresenta il bene messo in vendita dall’utente standard sulla bacheca. Gli attributi che definiscono un annuncio sono: un codice identificativo, il titolo del bene, la descrizione del bene, il prezzo, la data di pubblicazione e un booleano “venduto” che indica se il bene è stato venduto o no



Inserimento e cardinalità della relazione “Inserimento”:

E' la relazione che unisce le due entità “Utente_Standard” e “Annuncio”. La cardinalità di “Utente_Standard” è (0,N) perché un utente può non aver ancora inserito annunci o averne inseriti molti, mentre per “Annuncio” si ha una cardinalità (1,1) perché un annuncio è inserito da un solo utente.



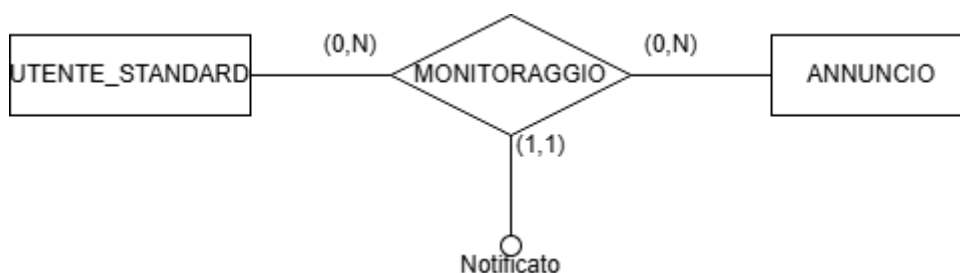
Inserimento e cardinalità della relazione “Appartenenza”:

La relazione “Appartenenza” lega le due entità “Categoria” e “Annuncio”. La cardinalità è di (1,1) per “Annuncio” perché un annuncio deve selezionare esattamente una categoria, mentre la cardinalità di “Categoria” è di (0,N) poiché una categoria può non avere annunci o averne molti.



Inserimento e cardinalità della relazione “Monitoraggio”:

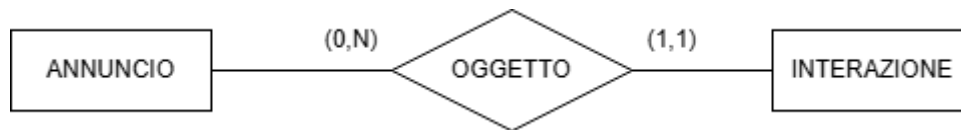
E' la relazione che lega le due entità “Utente_Standard” e “Annuncio”. Andrò a mettere un attributo “Notificato” con cardinalità (1,1) che notifica all’utente se l’annuncio è stato modificato o no. Entrambe le entità hanno cardinalità (0,N) poiché un utente può monitorare zero o molti annunci e l’annuncio può essere monitorato da zero o molti utenti.



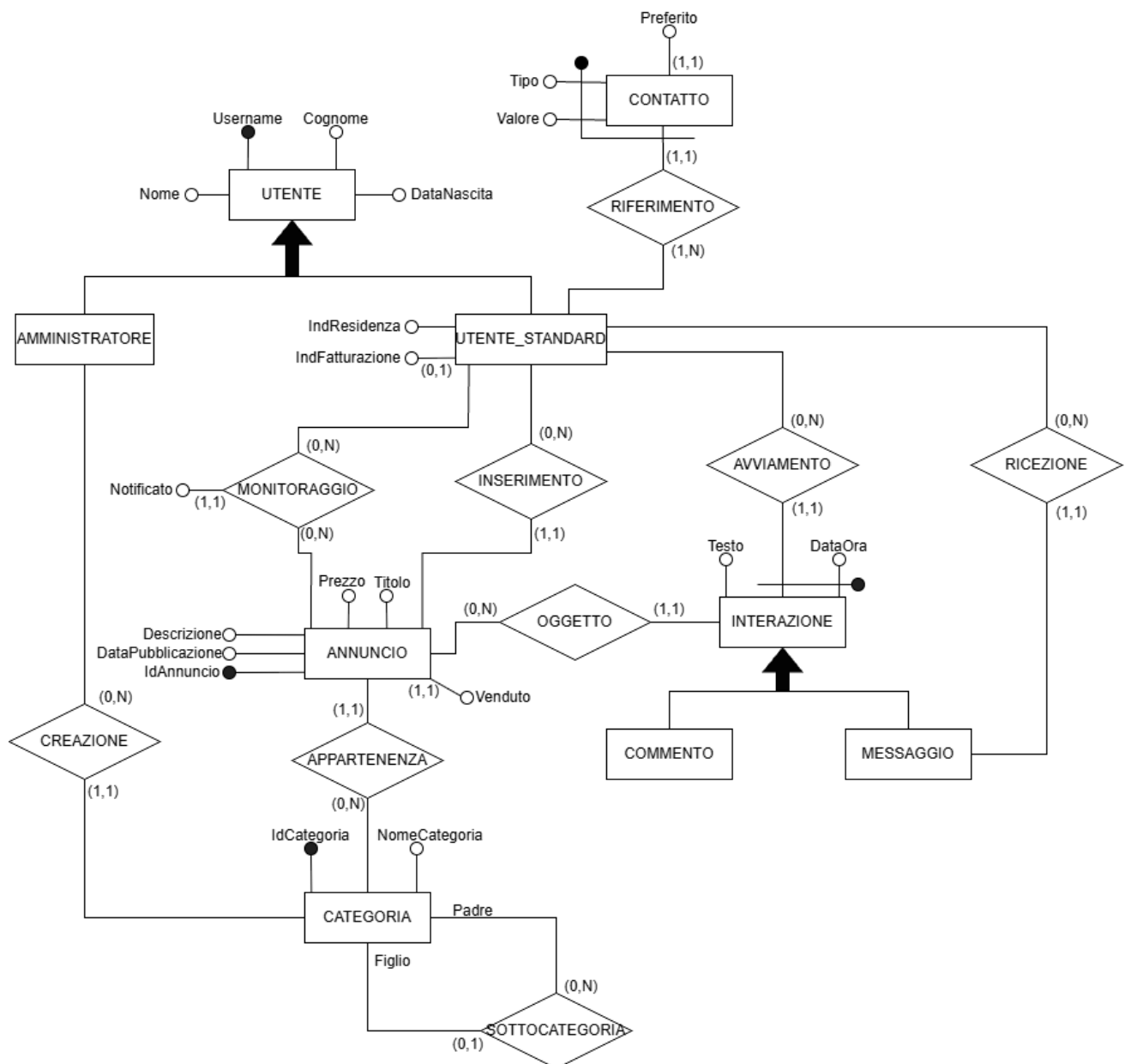
Inserimento e cardinalità della relazione “Oggetto”:

La relazione “Oggetto” collega l’entità “Interazione” all’entità “Annuncio”, definendo il contesto tematico e commerciale a cui si riferisce ogni interazione. La cardinalità di “Interazione” è (1,1) perché ogni interazione deve riferirsi obbligatoriamente a uno e un solo specifico annuncio, mentre quella di “Annuncio” è di (0,N) perché un annuncio appena pubblicato può non aver ancora ricevuto

alcun riscontro, oppure può accumulare nel corso del tempo un numero indefinito di commenti pubblici e messaggi privati.



Integrazione finale



Regole aziendali

(RV1) Un utente standard deve avere almeno un metodo di contatto memorizzato e esattamente uno solo deve risultare contrassegnato come preferito.

(RV2) Un annuncio contrassegnato con stato "Venduto" non può più subire modifiche al titolo, alla descrizione o al prezzo.

(RV3) Gli annunci pubblicati devono fare riferimento a una e una sola categoria all'interno della gerarchia.

(RV4) Un annuncio può ricevere commenti o messaggi privati solo se il suo stato non è "Venduto".

(RV5) In ogni messaggio privato inviato, almeno uno dei due partecipanti (mittente tramite Avviamento o destinatario tramite Ricezione) deve coincidere con l'utente autore dell'annuncio associato.

(RV6) Un utente standard non può inviare messaggi privati a se stesso.

(RV7) Un utente standard non può inserire commenti pubblici su annunci pubblicati da se stesso.

(RV8) Una categoria non può essere definita come sottocategoria di se stessa, né direttamente né indirettamente lungo la gerarchia dei padri.

(RV9) Non è consentito eliminare un metodo di contatto contrassegnato come preferito a meno che un altro contatto dello stesso utente non sia già stato preliminarmente designato come nuovo preferito.

(RD1) Se durante la registrazione l'indirizzo di fatturazione non viene specificato dall'utente, il suo valore viene ricavato e impostato automaticamente uguale all'indirizzo di residenza.

(RD2) Gli amministratori possono generare un report che riporti, per ciascun utente, la percentuale di annunci pubblicati contrassegnati come venduti.

(RD3) Quando un annuncio monitorato subisce una modifica, il flag "Notificato" nella relazione Monitoraggio viene impostato automaticamente a True.

(RD4) Quando un utente consulta gli annunci monitorati o ne legge gli aggiornamenti, il relativo flag "Notificato" nella relazione Monitoraggio viene reimpostato automaticamente al valore False.

Dizionario dei dati

Entità	Descrizione	Attributi	Identificatori
Utente	Soggetto registrato all'interno del sito.	Username, Nome, Cognome, Data di nascita	Username
Utente_Standard	Utente registrato della piattaforma abilitato ad acquistare o vendere beni usati attraverso la pubblicazione, consultazione e interazione sugli annunci.	Username, Nome, Cognome, Data di nascita, IndResidenza, IndFatturazione	Username
Amministratore	Utente con privilegi gestionali per la creazione della gerarchia delle categorie e l'elaborazione dei report.	Username, Nome, Cognome, Data di nascita	Username
Contatto	Recapito fornito dall'utente per la	Tipo, Valore,	Username, Tipo,

	reperibilità e le comunicazioni.	Preferito	Valore
Categoria	Sezione tematica della bacheca organizzata gerarchicamente per classificare gli annunci.	IdCategoria, NomeCategoria	IdCategoria
Annuncio	Inserzione pubblicata da un utente standard per la vendita di un bene usato.	IdAnnuncio, Titolo, Descrizione, Prezzo, DataPubblicazione, Venduto	IdAnnuncio
Interazione	Comunicazione scambiata da un utente in riferimento a uno specifico annuncio.	DataOra, Testo, IdAnnuncio	Username, DataOra
Commento	Interazione pubblica visibile al di sotto dell'annuncio del bene.	DataOra, Testo, IdAnnuncio	Username, DataOra
Messaggio	Interazione privata scambiata tra due utenti per accordarsi sulla compravendita.	DataOra, Testo, IdAnnuncio	Username, DataOra
Monitoraggio	Permette a un utente di seguire un annuncio per essere avvisato in caso di modifiche.	Notificato	Username, IdAnnuncio

4. Progettazione logica

Volume dei dati

Si suppone che il sistema debba gestire e archiviare i dati operativi su un arco temporale di 5 anni. Si ipotizzano circa 120 nuove registrazioni giornaliere di utenti standard, per un totale di circa 219.000 iscrizioni in 5 anni. Considerando un tasso di abbandono del 10%, si stimano 200.000 utenti standard attivi al termine del periodo.

Per quanto riguarda il personale di gestione, il sistema supporta la registrazione di nuovi account amministrativi nel corso del tempo; stimando l'ingresso medio di 1-2 amministratori al mese, si prevede un totale di 100 amministratori. L'entità Utente conta quindi complessivamente 200.100 istanze. Si assume che ciascun utente standard memorizzi in media 2 recapiti, portando a un volume stimato di 400.000 istanze per l'entità Contatto e per la relazione Riferimento. Si prevede che gli amministratori inseriscano un totale di 250 categorie. Di conseguenza, anche la relazione Creazione conterrà 250 istanze. Tra queste categorie, si stima che 220 siano sottocategorie associate a un'altra categoria tramite la relazione Sottocategoria. Si stima che ogni utente standard pubblichi mediamente 4 inserzioni nell'arco dei 5 anni, determinando un volume complessivo di 800.000 istanze per l'entità Annuncio, per la relazione Inserimento e per la relazione Appartenenza. Si ipotizza inoltre che circa il 60% degli utenti attivi segua annunci altrui, monitorando mediamente 3 inserzioni a testa, per un totale di 360.000 occorrenze nella relazione Monitoraggio. Si stima una media di 15 interazioni complessive per ciascun annuncio pubblicato (suddivise in 3 commenti pubblici e 12 messaggi privati con la relativa relazione Ricezione), determinando un volume complessivo di 12.000.000 di istanze per l'entità Interazione e per le relazioni di collegamento Avviamento e Oggetto, con 2.400.000 istanze per Commento e 9.600.000 istanze per Messaggio e Ricezione.

Concetto nello schema	Tipo ¹	Volume atteso
Utente	E	200.100
Utente_Standard	E	200.000
Amministratore	E	100
Contatto	E	400.000
Riferimento	R	400.000
Categoria	E	250
Creazione	R	250
Sottocategoria	R	220
Annuncio	E	800.000

¹ Indicare con E le entità, con R le relazioni

Inserimento	R	800.000
Appartenenza	R	800.000
Monitoraggio	R	360.000
Interazione	E	12.000.000
Avviamento	R	12.000.000
Oggetto	R	12.000.000
Commento	E	2.400.000
Messaggio	E	9.600.000
Ricezione	R	9.600.000

Tavola delle operazioni

Cod.	Descrizione	Frequenza attesa
OP1	Registrazione utente standard	120/giorno
OP2	Registrazione amministratore	2/mese
OP3	Login	25.000/giorno
OP4	Pubblicazione annuncio	450/giorno
OP5	Modifica annuncio attivo	150/giorno
OP6	Contrassegno annuncio come venduto	180/giorno
OP7	Visualizzazione bacheca annunci attivi (non venduti)	35.000/giorno
OP8	Visualizzazione annunci pubblicati dal singolo utente	2.500/giorno
OP9	Attivazione monitoraggio annuncio	300/giorno
OP10	Visualizzazione annunci monitorati e verifica notifiche	4.000/giorno
OP11	Reset flag notifica di monitoraggio	1.800/giorno
OP12	Invio commento pubblico	1.300/giorno
OP13	Visualizzazione commenti di un annuncio	15.000/giorno
OP14	Invio messaggio privato	5.000/giorno
OP15	Visualizzazione messaggi privati di un annuncio	8.000/giorno
OP16	Registrazione nuovo contatto di un utente	100/giorno
OP17	Rimozione contatto di un utente	20/giorno
OP18	Impostazione contatto preferito	50/giorno
OP19	Creazione nuova categoria	50/anno
OP20	Visualizzazione albero delle categoria	1.200/giorno
OP21	Generazione report vendite per utente	15/settimana

Costo delle operazioni

Il costo viene calcolato considerando le operazioni principali del sistema, assumendo come da specifica che un accesso in scrittura valga il doppio rispetto a uno in lettura:

- **OP1:** Prevede 1 lettura su Utente per verificare l'univocità dello username e 3 scritture (su Utente_Standard, su Contatto e sulla relazione Riferimento). Costo: $(1L+3S) * f = (1 + 3 * 2) * 120/\text{giorno} = 7 * 120 = 840 \text{ accessi/giorno}$.
- **OP2:** Prevede 1 lettura su Utente per verificare che lo username non sia occupato e 1 scrittura su Amministratore. Costo: $(1L + 1S) * f = (1 + 1 * 2) * 2/\text{mese} = 3 * 2 = 6 \text{ accessi/mese}$.
- **OP3:** Prevede 1 singolo accesso in lettura su Utente per autenticare le credenziali e individuare il ruolo. Costo: $1L * f = 1 * 25.000/\text{giorno} = 25.000 \text{ accessi/giorno}$.
- **OP4:** Prevede 1 accesso in lettura su Utente_Standard per accertare l'autore, 1 in lettura su Categoria per verificare l'esistenza della categoria, 1 in scrittura su Annuncio e 2 in scrittura per le relazioni Inserimento e Appartenenza. Costo: $(2L + 3S) * f = (2 + 3 * 2) * 450/\text{giorno} = 8 * 450 = 3.600 \text{ accessi/giorno}$.
- **OP5:** Prevede 1 accesso in lettura su Annuncio per verificare lo stato e l'autore, 1 in scrittura per aggiornare i campi dell'annuncio e, in presenza di monitoraggi attivi (stimando mediamente 3 utenti interessati), 3 scritture sulla relazione Monitoraggio per aggiornare il flag di notifica. Costo: $(1L + 4S) * f = (1 + 4 * 2) * 150/\text{giorno} = 9 * 150 = 1.350 \text{ accessi/giorno}$.
- **OP6:** Prevede 1 accesso in lettura su Annuncio per verificare autore e stato invenduto, 1 in scrittura su Annuncio per impostare Venduto = 1 e la cancellazione di eventuali record collegati (stimando mediamente 3 scritture su Monitoraggio). Costo: $(1L + 4S) * f = (1 + 4 * 2) * 180/\text{giorno} = 9 * 180 = 1.620 \text{ accessi/giorno}$.
- **OP7:** Prevede la consultazione del catalogo pubblico delle inserzioni non vendute: stimando una consultazione media di 20 annunci per pagina con relativa categoria, si hanno 20 accessi in lettura su Annuncio e 20 su Categoria. Costo: $(20L + 20L) * f = 40 * 35.000/\text{giorno} = 1.400.000 \text{ accessi/giorno}$.
- **OP8:** Prevede 1 lettura su Utente_Standard per identificare l'utente e, considerando una media di 4 annunci per utente con relativa categoria, 4 letture su Annuncio e 4 letture su Categoria. Costo: $(1L + 4L + 4L) * f = 9 * 2.500/\text{giorno} = 22.500 \text{ accessi/giorno}$.
- **OP9:** Prevede 1 accesso in lettura su Utente_Standard, 1 in lettura su Annuncio per accertare che non sia venduto né di proprietà dell'utente stesso, e 1 in scrittura sulla relazione Monitoraggio. Costo: $(2L + 1S) * f = (2 + 1 * 2) * 300/\text{giorno} = 4 * 300 = 1.200 \text{ accessi/giorno}$.

- **OP10:** Prevede 1 lettura su Utente_Standard e, considerando una media di 3 annunci seguiti per utente con relativa categoria, 3 letture su Monitoraggio, 3 su Annuncio e 3 su Categoria. Costo: $(1L + 3L + 3L + 3L) * f = 10 * 4.000/\text{giorno} = 40.000 \text{ accessi/giorno}$.
- **OP11:** Prevede 1 accesso in lettura su Monitoraggio per individuare le notifiche pendenti e, stimando mediamente 2 record da azzerare, 2 accessi in scrittura su Monitoraggio. Costo: $(1L + 2S) * f = (1 + 2 * 2) * 1.800/\text{giorno} = 5 * 1.800 = 9.000 \text{ accessi/giorno}$.
- **OP12:** Prevede 1 accesso in lettura su Utente_Standard, 1 in lettura su Annuncio per accertare che non sia venduto né dell'autore stesso, 1 in scrittura su Commento e 2 in scrittura sulle relazioni Avviamento e Oggetto. Costo: $(2L + 3S) * f = (2 + 3 * 2) * 1.300/\text{giorno} = 8 * 1.300 = 10.400 \text{ accessi/giorno}$.
- **OP13:** Prevede 1 lettura su Annuncio e, stimando una media di 3 commenti per inserzione, 3 letture su Commento e 3 letture sulla relazione Avviamento per estrarre l'autore di ciascun commento. Costo: $(1L + 3L + 3L) * f = 7 * 15.000/\text{giorno} = 105.000 \text{ accessi/giorno}$.
- **OP14:** Prevede 2 accessi in lettura su Utente_Standard (mittente e destinatario), 1 in lettura su Annuncio per accertare stato e titolarità dell'autore, 1 in scrittura su Messaggio e 3 in scrittura sulle relazioni Avviamento, Oggetto e Ricezione. Costo: $(3L + 4S) * f = (3 + 4 * 2) * 5.000/\text{giorno} = 11 * 5.000 = 55.000 \text{ accessi/giorno}$.
- **OP15:** Prevede 1 lettura su Annuncio e, stimando una media di 4 messaggi scambiati tra le due parti per la conversazione selezionata, 4 letture su Messaggio, 4 su Avviamento e 4 sulla relazione Ricezione. Costo: $(1L + 4L + 4L + 4L) * f = 13 * 8.000/\text{giorno} = 104.000 \text{ accessi/giorno}$.
- **OP16:** Prevede 1 accesso in lettura su Utente_Standard, 1 in scrittura su Contatto e 1 in scrittura sulla relazione Riferimento. Costo: $(1L + 2S) * f = (1 + 2 * 2) * 100/\text{giorno} = 5 * 100 = 500 \text{ accessi/giorno}$.
- **OP17:** Prevede 1 accesso in lettura su Contatto per verificare che non sia l'unico né quello preferito, 1 in scrittura per cancellare il recapito da Contatto e 1 in scrittura per rimuovere il legame in Riferimento. Costo: $(1L + 2S) * f = (1 + 2 * 2) * 20/\text{giorno} = 5 * 20 = 100 \text{ accessi/giorno}$.
- **OP18:** Prevede 1 accesso in lettura su Contatto per verificare l'esistenza del recapito, 1 in scrittura per azzerare il vecchio preferito e 1 in scrittura per contrassegnare il nuovo preferito. Costo: $(1L + 2S) * f = (1 + 2 * 2) * 50/\text{giorno} = 5 * 50 = 250 \text{ accessi/giorno}$.

- **OP19:** Prevede 1 accesso in lettura su Amministratore per verificare i privilegi, 1 in lettura sulla categoria genitore eventuale, 1 in scrittura su Categoria e 1 in scrittura sulla relazione Creazione. Costo: $(2L + 2S) * f = (2 + 2 * 2) * 50/\text{anno} = 6 * 50 = 300$ accessi/anno.
- **OP20:** Prevede la consultazione dell'elenco completo delle categorie: considerando la lettura di tutte le 250 categorie e il recupero del relativo genitore per le 220 sottocategorie esistenti, si hanno 250 accessi in lettura sull'entità Categoria e 220 sulla relazione Sottocategoria. Costo: $(250L + 220L) * f = 470 * 1.200/\text{giorno} = 564.000$ accessi/giorno.
- **OP21:** Prevede 1 accesso in lettura su Amministratore per validare i permessi operativi, 1 lettura sequenziale aggregata su Utente_Standard e 1 scansione aggregata su Annuncio per quantificare il totale delle inserzioni pubblicate e vendute. Costo: $(1L + 1L + 1L) * f = 3 * 15/\text{settimana} = 45$ accessi/settimana.

Ristrutturazione dello schema E-R

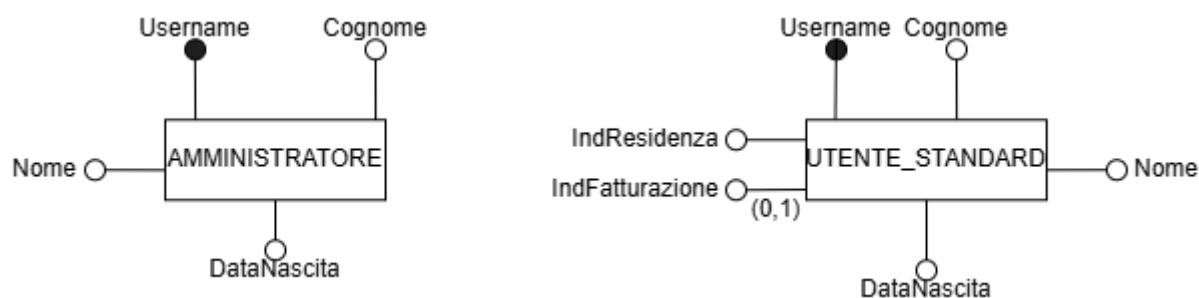
Analisi delle ridondanze

Dall'analisi della progettazione concettuale e dei requisiti operativi non si riscontrano dati o attributi ridondanti. Tutti gli attributi presenti nelle entità e nelle relazioni memorizzano informazioni primitive indispensabili per il funzionamento della bacheca. Pertanto, non si rende necessaria alcuna eliminazione di attributi o tabelle per motivi di ridondanza.

Eliminazione delle generalizzazioni

- **Generalizzazione entità “Utente”:** Si opta per l'accorpamento del genitore nelle entità figlie. L'entità Utente non possiede relazioni dirette autonome al di fuori di quelle ereditate. Inoltre, le entità figlie partecipano a insiemi di relazioni completamente disgiunti: Utente_Standard interagisce con contatti, annunci, commenti, messaggi e monitoraggi, mentre Amministratore interagisce unicamente con categorie e reportistica. Mantenere due entità distinte evita una tabella sovradimensionata ricca di valori nulli per i campi di residenza e fatturazione.

L'autenticazione comune e la condivisione della chiave di accesso verranno gestite a livello logico/fisico demandando la persistenza dei dati di login all'entità di supporto Credenziali.



- **Generalizzazione entità “Interazione”:** Anche in questo caso si sceglie l'accorpamento dell'entità genitore nelle entità figlie. Sebbene entrambe condividano gli attributi di base (DataOra e Testo), esse assolvono a compiti operativi differenti: Messaggio gestisce comunicazioni private bidirezionali tra utenti attraverso la relazione Ricezione, mentre Commento rappresenta un'osservazione pubblica rivolta all'inserzione. Con la scomparsa dell'entità Interazione, l'entità Commento eredita gli attributi comuni e si collega direttamente alla relazione Oggetto verso l'entità Annuncio, scelta che verrà mostrata graficamente direttamente nello schema E-R ristrutturato. L'accorpamento nelle figlie elimina la necessità di interrogare una tabella comune generalizzata con frequenti controlli di tipo e riduce lo spazio occupato.

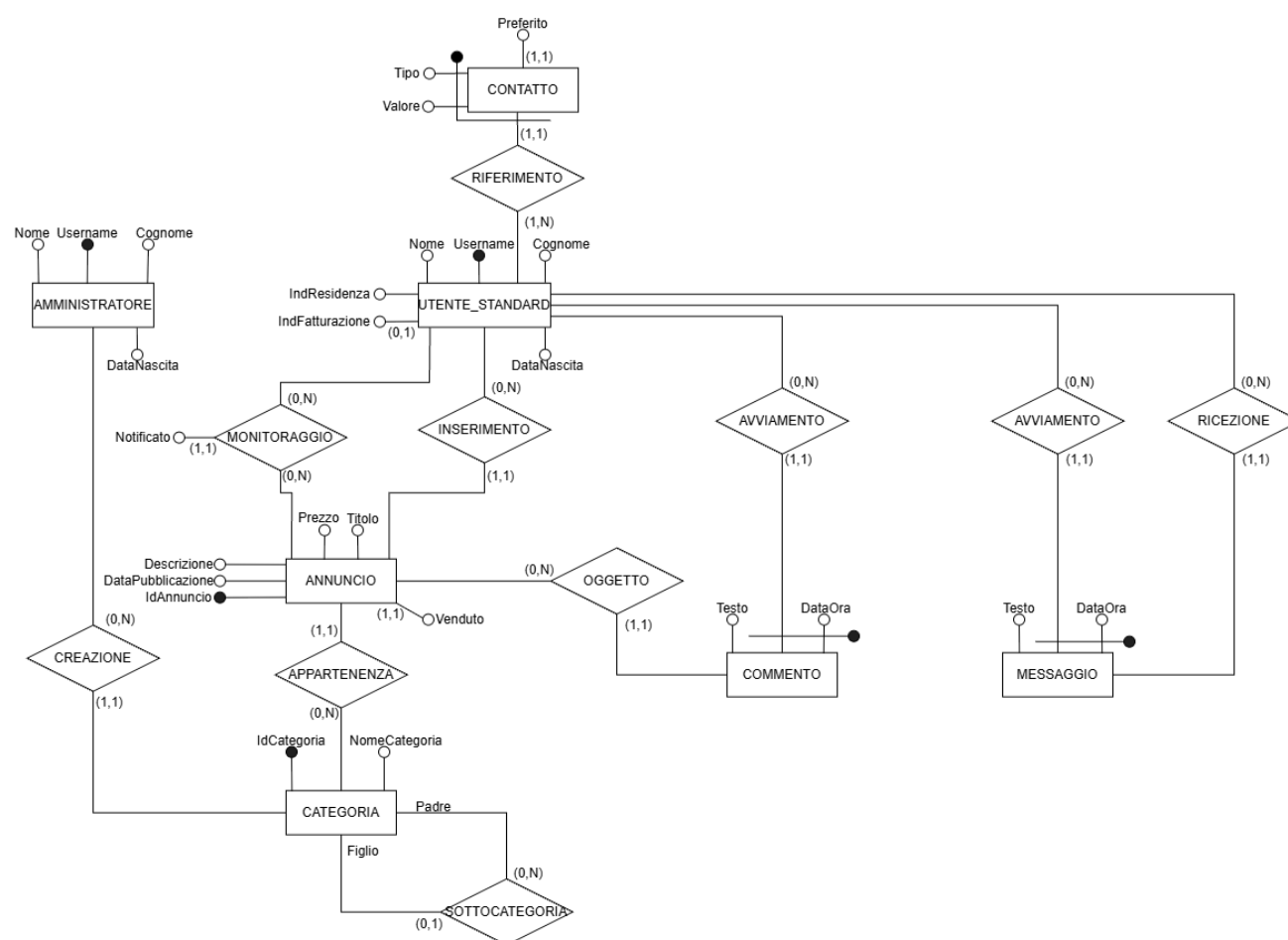
Eliminazione degli attributi multivalore

Nella progettazione concettuale non sono presenti attributi multivalore.

Scelta degli identificatori primari

Per la scelta degli identificatori primari si confermano le decisioni prese in fase di progettazione concettuale.

Schema E-R ristrutturato



Trasformazione di attributi e identificatori

All'interno dello schema concettuale ristrutturato sono presenti diversi identificatori esterni che richiedono una ridenominazione funzionale per evitare ambiguità nella successiva traduzione logica e distinguere chiaramente i ruoli assunti dalle entità all'interno delle relazioni:

- **Username:** costituisce l'identificatore primario originario dell'entità Utente_Standard. Viene ereditato come identificatore esterno nelle seguenti entità:
 - In Contatto, dove fa parte della chiave primaria composta per identificare l'utente titolare del recapito telefonico o telematico.
 - In Commento, dove fa parte della chiave primaria composta insieme all'attributo DataOra e viene ridenominato in Commentatore per indicare esplicitamente l'utente che inserisce il commento.
 - In Messaggio, dove fa parte della chiave primaria composta con DataOra e viene ridenominato in Mittente per identificare l'utente che avvia la conversazione privata. Inoltre, tramite la relazione Ricezione, compare un

secondo riferimento a Username ridenominato in Destinatario per indicare l'utente che riceve il messaggio privato.

- Nella tabella di associazione Monitoraggio, dove compare all'interno della chiave primaria composta per specificare quale utente segue le modifiche dell'annuncio.
- **IdAnnuncio:** costituisce l'identificatore primario univoco dell'entità Annuncio. Viene propagato ed ereditato all'interno del modello nei seguenti casi:
 - Nella tabella di associazione Monitoraggio, dove compare come parte della chiave primaria composta insieme a Username per identificare univocamente la singola inserzione seguita da quello specifico utente.
 - In Commento, dove viene ereditato come chiave esterna tramite la relazione Oggetto per associare ciascun commento pubblico all'annuncio del bene a cui si riferisce.
 - In Messaggio, dove viene ereditato come chiave esterna tramite la relazione Oggetto per legare la trattativa privata tra mittente e destinatario allo specifico annuncio in compravendita.

Traduzione di entità e associazioni

In questo paragrafo illustriamo il passaggio dal modello concettuale a quello relazionale: a questo scopo, ogni entità viene trasformata in una tabella mantenendo i propri attributi e le chiavi primarie, mentre le relazioni tra entità vengono tradotte tramite l'uso di chiavi esterne o con tabelle apposite per i legami molti a molti. Si introduce inoltre la relazione Credenziali per supportare l'autenticazione centralizzata e la persistenza sicura delle credenziali di accesso di entrambi i profili utente. Quindi avremo:

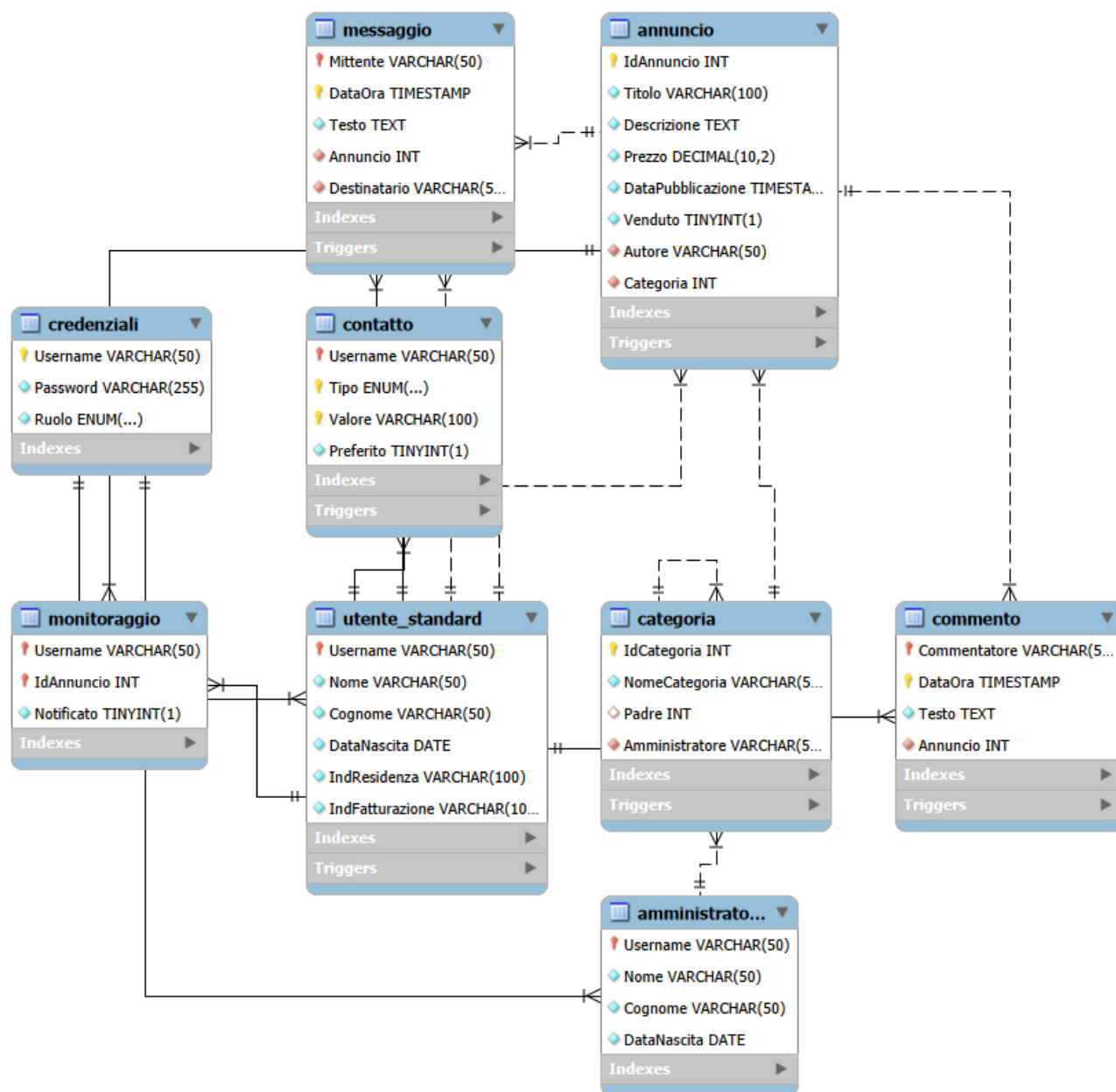
- Credenziali (Username, Password, Ruolo)
- Utente_Standard (Username*, Nome, Cognome, DataNascita, IndResidenza, IndFatturazione)
- Amministratore (Username*, Nome, Cognome, DataNascita)
- Contatto (Username, Tipo, Valore, Preferito)
- Categoria (IdCategoria, NomeCategoria, Padre*, Amministratore*)
- Annuncio (IdAnnuncio, Titolo, Descrizione, Prezzo, DataPubblicazione, Venduto, Autore*, Categoria*)
- Commento (Commentatore*, DataOra, Testo, Annuncio*)
- Messaggio (Mittente*, DataOra, Testo, Destinatario*, Annuncio*)

- Monitoraggio (Username*, IdAnnuncio*, Notificato)

Vincoli di integrità referenziale:

- Utente_Standard (Username) $\subseteq$ Credenziali (Username)
- Amministratore (Username) $\subseteq$ Credenziali (Username)
- Contatto (Username) $\subseteq$ Utente_Standard (Username)
- Categoria (Padre) $\subseteq$ Categoria (IdCategoria)
- Categoria (Amministratore) $\subseteq$ Amministratore (Username)
- Annuncio (Autore) $\subseteq$ Utente_Standard (Username)
- Annuncio (Categoria) $\subseteq$ Categoria (IdCategoria)
- Commento (Commentatore) $\subseteq$ Utente_Standard (Username)
- Commento (Annuncio) $\subseteq$ Annuncio (IdAnnuncio)
- Messaggio (Mittente) $\subseteq$ Utente_Standard (Username)
- Messaggio (Destinatario) $\subseteq$ Utente_Standard (Username)
- Messaggio (Annuncio) $\subseteq$ Annuncio (IdAnnuncio)
- Monitoraggio (Username) $\subseteq$ Utente_Standard (Username)
- Monitoraggio (IdAnnuncio) $\subseteq$ Annuncio (IdAnnuncio)

Ora forniamo una rappresentazione grafica del modello relazionale completo:



Normalizzazione del modello relazionale

Il modello relazionale precedentemente descritto è in Forma Normale di Boyce-Codd (BCNF). Infatti, per ogni dipendenza funzionale non banale $X \rightarrow Y$ definita sulle relazioni dello schema, l'insieme di attributi determinante X costituisce una chiave candidata o una superchiave. Analizziamo nel dettaglio ciascuna relazione:

- Per **Credenziali** la dipendenza funzionale presente è:

Username $\rightarrow$ Password, Ruolo

La relazione rispetta la BCNF poiché l'attributo determinante **Username** è la superchiave.

- Per **Utente_Standard** la dipendenza funzionale associata è:

Username → **Nome, Cognome, DataNascita, IndResidenza, IndFatturazione**

La relazione soddisfa la BCNF in quanto **Username** è superchiave della tabella.

- Per **Amministratore** l'analisi è analoga a quella svolta per **Utente_Standard**:

Username → **Nome, Cognome, DataNascita**

Dato che **Username** è superchiave, la relazione rispetta pienamente la BCNF.

- Per **Contatto** la dipendenza funzionale è:

Username, Tipo, Valore → **Preferito**

La relazione rispetta la BCNF poiché il determinante a sinistra coincide esattamente con la chiave primaria composta, risultando una superchiave.

- Per **Categoria** la dipendenza funzionale è:

IdCategoria → **NomeCategoria, Padre, Amministratore**

IdCategoria è superchiave e quindi la relazione rispetta pienamente la BCNF.

- Per **Annuncio** la dipendenza funzionale è:

IdAnnuncio → **Titolo, Descrizione, Prezzo, DataPubblicazione, Venduto, Autore, Categoria**

La relazione è in BCNF poiché **IdAnnuncio** rappresenta la chiave primaria ed è pertanto superchiave.

- Per **Commento** la dipendenza funzionale è:

Commentatore, DataOra → **Testo, Annuncio**

La coppia composta da **Commentatore** e **DataOra** costituisce la chiave primaria della tabella; essendo superchiave, la relazione rispetta la BCNF.

- Per **Messaggio** la dipendenza funzionale è:

Mittente, DataOra → **Testo, Destinatario, Annuncio**

Anche in questo caso la chiave primaria composta (**Mittente, DataOra**) è superchiave e garantisce la conformità alla BCNF.

- Per **Monitoraggio** la dipendenza funzionale è:

Username, IdAnnuncio → **Notificato**

A sinistra della freccia abbiamo la chiave primaria composta (**Username, IdAnnuncio**), che è superchiave della tabella; quindi la relazione rispetta pienamente la BCNF.

Non essendo presenti dipendenze funzionali parziali né dipendenze transitive su attributi non chiave, lo schema relazionale risulta privo di anomalie di inserimento, cancellazione o modifica, rendendo superflue ulteriori decomposizioni.

5. Progettazione fisica

Utenti e privilegi

La gestione della sicurezza all'interno del sistema informatico si fonda su un controllo granulare degli accessi e sulla persistenza dell'identità dell'utente lungo l'intera sessione applicativa. All'atto dell'autenticazione, il livello software recupera lo username associato alle credenziali verificate e lo impiega come parametro identificativo cardine per tutte le chiamate successive. Tale approccio sfrutta la natura di chiave primaria dell'attributo username nelle relazioni credenziali, utente standard e amministratore, azzerando l'esigenza di interrogazioni ridondanti per ricavare i dati dell'operatore e garantendo un legame univoco tra la sessione attiva e le modifiche persistite sul database.

L'infrastruttura prevede due profili operativi distinti, ciascuno associato a un'utenza dedicata configurata a livello di DBMS: l'utente standard (`app_standard@localhost`) e l'amministratore (`app_admin@localhost`).

Per scongiurare manomissioni dirette dello stato persistente e tutelare l'integrità referenziale, è stato applicato rigorosamente il principio del minimo privilegio (Principle of Least Privilege). A nessuna delle due utenze applicative viene concessa la visibilità diretta sulle tabelle fisiche o sulle viste mediante autorizzazioni di tipo `select`, `insert`, `update` o `delete`. L'intero layer dei dati risulta incapsulato dalle routine transazionali create dall'amministratore del DBMS (`root@localhost`): i client interagiscono con la base di dati unicamente mediante il permesso di esecuzione sulle specifiche stored procedure abilitate per ciascun ruolo.

La ripartizione dei privilegi operativi è così articolata:

- **Fase di Accesso e Autenticazione:** Sia `app_standard` che `app_admin` dispongono del diritto `EXECUTE` sulla procedura `login`. Questa routine confronta la password immessa con quella registrata a sistema ed estrae il ruolo associato, consentendo all'applicazione di instradare il corretto flusso operativo in base alle autorizzazioni del profilo.
- **Profilo Utente Standard:** Possiede i privilegi di esecuzione indispensabili per usufruire delle funzionalità operative della bacheca. Nello specifico, può eseguire le procedure per l'iscrizione anagrafica e la gestione completa del ciclo di vita dei recapiti personali (`registrazione_utente_standard`, `registrazione_contatto`, `imposta_contatto_preferito`, `rimuovi_contatto`), la gestione del ciclo di vita dei propri annunci (`pubblicazione_annuncio`, `modifica_annuncio`, `annuncio_venduto`), l'invio di messaggi e commenti (`invio_commento`, `invio_messaggio`), il monitoraggio delle inserzioni con verifica e azzeramento delle notifiche (`monitora_annuncio`, `visualizza_annunci_monitorati`, `verifica_annunci_modificati`,

resetta_flag_notifica) e la consultazione dei dati pubblici o del proprio storico (visualizza_annunci_non_venduti, visualizza_annunci_utente, visualizza_commenti_annuncio, visualizza_messaggi_annuncio, visualizza_tutte_categorie).

- **Profilo Amministratore:** Gode esclusivamente dei diritti esecutivi legati alla gestione amministrativa del sistema. L'account è autorizzato a invocare la procedura di accreditamento di nuovi profili gestionali (registrazione_amministratore), la strutturazione dell'albero delle categorie (crea_categoria, visualizza_tutte_categorie) e l'elaborazione dei dati aggregati d'andamento delle vendite (genera_report_vendite). Tale profilo non interviene sulle trattative né partecipa alle comunicazioni tra privati, preservando la netta separazione dei compiti tra gestione della piattaforma e operatività degli utenti.

Strutture di memorizzazione

Tabella <Credenziali>		
Colonna	Tipo di dato	Attributi ²
Username	VARCHAR(50)	PK, NN
Password	VARCHAR(255)	NN
Ruolo	ENUM('standard', 'amministratore')	NN

Tabella <Utente_Standard>		
Colonna	Tipo di dato	Attributi
Username	VARCHAR(50)	PK, NN, FK
Nome	VARCHAR(50)	NN
Cognome	VARCHAR(50)	NN
DataNascita	DATE	NN
IndResidenza	VARCHAR(100)	NN
IndFatturazione	VARCHAR(100)	NN

Tabella <Amministratore>		
Colonna	Tipo di dato	Attributi
Username	VARCHAR(50)	PK, NN, FK
Nome	VARCHAR(50)	NN
Cognome	VARCHAR(50)	NN
DataNascita	DATE	NN

Tabella <Contatto>		
Colonna	Tipo di dato	Attributi
Username	VARCHAR(50)	PK, NN, FK

² PK = primary key, NN = not null, UQ = unique, UN = unsigned, AI = auto increment, FK = foreign key. È ovviamente possibile specificare più di un attributo per ciascuna colonna.

Tipo	ENUM('email','telefono','cellulare')	PK, NN
Valore	VARCHAR(100)	PK, NN
Preferito	TINYINT(1)	NN

Tabella <Categoria>		
Colonna	Tipo di dato	Attributi
IdCategoria	INT	PK, NN, AI
NomeCategoria	VARCHAR(50)	NN, UQ
Padre	INT	FK
Amministratore	VARCHAR(50)	NN, FK

Tabella <Annuncio>		
Colonna	Tipo di dato	Attributi
IdAnnuncio	INT	PK, NN, AI
Titolo	VARCHAR(100)	NN
Descrizione	TEXT	NN
Prezzo	DECIMAL(10,2)	NN, UN
DataPubblicazione	TIMESTAMP	NN
Venduto	TINYINT(1)	NN
Autore	VARCHAR(50)	NN, FK
Categoria	INT	NN, FK

Tabella <Commento>		
Colonna	Tipo di dato	Attributi
Commentatore	VARCHAR(50)	PK, NN, FK
DataOra	TIMESTAMP	PK, NN
Testo	TEXT	NN
Annuncio	INT	NN, FK

Tabella <Messaggio>		
Colonna	Tipo di dato	Attributi
Mittente	VARCHAR(50)	PK, NN, FK
DataOra	TIMESTAMP	PK, NN
Testo	TEXT	NN
Annuncio	INT	NN, FK
Destinatario	VARCHAR(50)	NN, FK

Tabella <Monitoraggio>		
Colonna	Tipo di dato	Attributi
Username	VARCHAR(50)	PK, NN, FK
IdAnnuncio	INT	PK, NN, FK
Notificato	TINYINT(1)	NN

Indici

All'interno della base di dati non sono stati definiti indici secondari dedicati esclusivamente all'ottimizzazione prestazionale, al di fuori di quelli generati automaticamente dal sistema per la gestione delle chiavi primarie, dei vincoli di unicità e delle chiavi esterne.

Trigger

1) **default_fatturazione_insert**

Questo trigger imposta automaticamente il valore dell'indirizzo di fatturazione uguale a quello di residenza se non viene specificato in fase di registrazione.

```
CREATE DEFINER='root'@'localhost' TRIGGER `default_fatturazione_insert` BEFORE
INSERT ON `utente_standard` FOR EACH ROW
BEGIN
    IF NEW.IndFatturazione IS NULL OR TRIM(NEW.IndFatturazione) = ' ' THEN
        SET NEW.IndFatturazione = NEW.IndResidenza;
    END IF;
END
```

2) **blocco_annunci_venduti_update**

Questo trigger intercetta ogni tentativo di aggiornamento su un annuncio e blocca l'operazione se risulta già venduto.

```
CREATE DEFINER='root'@'localhost' TRIGGER `blocco_annunci_venduti_update`
BEFORE UPDATE ON `annuncio` FOR EACH ROW
BEGIN
    IF OLD.Venduto = 1 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Impossibile modificare un annuncio contrassegnato come
venduto.';
    END IF;
END
```

3) **notifica_modifica_annuncio_update**

Questo trigger rileva variazioni a titolo, descrizione o prezzo di un annuncio attivo e imposta a 1 il flag 'Notificato' per tutti gli utenti che lo monitorano.

```
CREATE DEFINER='root'@'localhost' TRIGGER `notifica_modifica_annuncio_update`
AFTER UPDATE ON `annuncio` FOR EACH ROW
```

```
BEGIN
    IF NEW.Venduto = 0 AND (OLD.Titolo <> NEW.Titolo OR OLD.Descrizione <>
NEW.Descrizione OR OLD.Prezzo <> NEW.Prezzo) THEN
        UPDATE `Monitoraggio`
        SET `Notificato` = 1
        WHERE `IdAnnuncio` = NEW.IdAnnuncio;
    END IF;
END
```

4) **rimuovi_monitoraggio_dopo_vendita**

Questo trigger elimina automaticamente tutte le righe di monitoraggio associate a un annuncio non appena questo passa allo stato di venduto.

```
CREATE DEFINER=`root`@`localhost` TRIGGER `rimuovi_monitoraggio_dopo_vendita`
AFTER UPDATE ON `annuncio` FOR EACH ROW
BEGIN
    IF NEW.Venduto = 1 AND OLD.Venduto = 0 THEN
        DELETE FROM `Monitoraggio`
        WHERE `IdAnnuncio` = NEW.IdAnnuncio;
    END IF;
END
```

5) **controllo_ciclicita_categoria_insert**

Questo trigger impedisce in fase di inserimento che una categoria venga creata impostando se stessa come categoria padre.

```
CREATE DEFINER=`root`@`localhost` TRIGGER `controllo_ciclicita_categoria_insert`
BEFORE INSERT ON `categoria` FOR EACH ROW
BEGIN
    IF NEW.Padre IS NOT NULL AND NEW.IdCategoria IS NOT NULL AND NEW.Padre
= NEW.IdCategoria THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Una categoria non puo essere padre di se stessa.';
    END IF;
END
```

6) **controllo_ciclicita_categoria_update**

Questo trigger risale a ritroso la catena gerarchica delle categorie prima di un aggiornamento, bloccando l'operazione se viene generato un ciclo diretto o indiretto.

```
CREATE DEFINER=`root`@`localhost` TRIGGER `controllo_ciclicita_categoria_update`  
BEFORE UPDATE ON `categoria` FOR EACH ROW  
BEGIN  
    DECLARE v_cursore_padre INT;  
  
    IF NEW.Padre IS NOT NULL AND NEW.Padre = NEW.IdCategoria THEN  
        SIGNAL SQLSTATE '45000'  
        SET MESSAGE_TEXT = 'Una categoria non puo essere padre di se stessa.';  
    END IF;  
  
    SET v_cursore_padre = NEW.Padre;  
    WHILE v_cursore_padre IS NOT NULL DO  
        IF v_cursore_padre = NEW.IdCategoria THEN  
            SIGNAL SQLSTATE '45000'  
            SET MESSAGE_TEXT = 'Rilevato ciclo nella gerarchia delle categorie.';  
        END IF;  
  
        SELECT `Padre` INTO v_cursore_padre  
        FROM `Categoria`  
        WHERE `IdCategoria` = v_cursore_padre;  
    END WHILE;  
END
```

7) **controllo_commento_insert**

Questo trigger blocca l'inserimento del commento se l'annuncio è stato già venduto o se l'autore dell'annuncio tenta di commentare la propria inserzione.

```
CREATE DEFINER=`root`@`localhost` TRIGGER `controllo_commento_insert` BEFORE  
INSERT ON `commento` FOR EACH ROW  
BEGIN  
    DECLARE v_autore VARCHAR(50);  
    DECLARE v_venduto TINYINT(1);  
  
    SELECT `Autore`, `Venduto` INTO v_autore, v_venduto  
    FROM `Annuncio`  
    WHERE `IdAnnuncio` = NEW.Annuncio;
```

```
IF v_venduto = 1 THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Impossibile commentare un annuncio già contrassegnato
come venduto.';
END IF;

IF NEW.Commentatore = v_autore THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'L autore dell annuncio non puo inserire commenti sui propri
annunci.';
END IF;
END
```

8) **controllo_contatto_delete**

Questo trigger impedisce l'eliminazione dell'unico recapito rimasto a un utente o di quello contrassegnato come preferito.

```
CREATE DEFINER=`root`@`localhost` TRIGGER `controllo_contatto_delete` BEFORE
DELETE ON `contatto` FOR EACH ROW
BEGIN
    DECLARE v_totale_contatti INT;

    SELECT COUNT(*) INTO v_totale_contatti
    FROM `Contatto`
    WHERE `Username` = OLD.Username;

    IF v_totale_contatti <= 1 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Impossibile eliminare l unico recapito di contatto
memorizzato.';
    END IF;

    IF OLD.Preferito = 1 THEN
        SIGNAL SQLSTATE '45000'
```

```
SET MESSAGE_TEXT = 'Impossibile eliminare il contatto preferito. Designare prima un altro preferito.';
```

```
END IF;
```

```
END
```

9) **controllo_messaggio_insert**

Questo trigger blocca l'invio di messaggi su annunci già venduti, vieta l'auto-invio tra lo stesso utente e verifica che almeno uno tra mittente e destinatario sia l'autore dell'annuncio.

```
CREATE DEFINER=`root`@`localhost` TRIGGER `controllo_messaggio_insert` BEFORE INSERT ON `messaggio` FOR EACH ROW
```

```
BEGIN
```

```
DECLARE v_autore VARCHAR(50);
```

```
DECLARE v_venduto TINYINT(1);
```

```
IF NEW.Mittente = NEW.Destinatario THEN
```

```
SIGNAL SQLSTATE '45000'
```

```
SET MESSAGE_TEXT = 'Un utente non puo inviare messaggi privati a se stesso.';
```

```
END IF;
```

```
SELECT `Autore`, `Venduto` INTO v_autore, v_venduto
```

```
FROM `Annuncio`
```

```
WHERE `IdAnnuncio` = NEW.Annuncio;
```

```
IF v_venduto = 1 THEN
```

```
SIGNAL SQLSTATE '45000'
```

```
SET MESSAGE_TEXT = 'Impossibile inviare messaggi per un annuncio gia venduto.';
```

```
END IF;
```

```
IF NEW.Mittente <> v_autore AND NEW.Destinatario <> v_autore THEN
```

```
SIGNAL SQLSTATE '45000'
```

```
SET MESSAGE_TEXT = 'Uno tra mittente e destinatario deve essere l autore dell annuncio.';
```

```
END IF;
```

```
END
```

Eventi

All'interno della base di dati sono stati implementati due eventi temporizzati schedulati, eseguiti periodicamente in background dal DBMS tramite il gestore dedicato previa abilitazione del parametro globale event_scheduler. Entrambi gli eventi vengono istanziati in modo statico e permanente in fase di configurazione dell'infrastruttura, senza ricorrere a istanziazioni effimere a runtime.

L'evento "elimina_messaggi_privati_vecchi" viene eseguito quotidianamente per cancellare le comunicazioni private memorizzate nella tabella Messaggio antecedenti a 2 anni rispetto al timestamp corrente.

L'evento "archivia_annunci_scaduti" viene eseguito ogni 24 ore per gestire il ciclo di vita degli annunci rimasti invenduti. Qualora un annuncio risulti ancora attivo a distanza di oltre un anno dalla data di pubblicazione, l'evento ne forza la chiusura impostando il flag Venduto a 1. Tale aggiornamento impedisce ulteriori modifiche, ne ritira la visibilità dalla vista bacheca_pubblica e azzerava automaticamente le registrazioni di monitoraggio pendenti tramite il trigger "rimuovi_monitoraggio_dopo_vendita".

```
USE `bacheca_annunci`;
```

```
SET GLOBAL event_scheduler = ON;
```

```
DELIMITER $$
```

```
DROP EVENT IF EXISTS `elimina_messaggi_privati_vecchi`$$
CREATE EVENT `elimina_messaggi_privati_vecchi`
ON SCHEDULE EVERY 1 DAY
STARTS CURRENT_TIMESTAMP
ON COMPLETION PRESERVE
COMMENT 'Pulizia periodica dei messaggi privati registrati da piu di 2 anni'
DO
BEGIN
    DELETE FROM `Messaggio`
    WHERE `DataOra` < (CURRENT_TIMESTAMP - INTERVAL 2 YEAR);
END$$
```

```
DROP EVENT IF EXISTS `archivia_annunci_scaduti`$$
CREATE EVENT `archivia_annunci_scaduti`
ON SCHEDULE EVERY 1 DAY
STARTS CURRENT_TIMESTAMP
ON COMPLETION PRESERVE
COMMENT 'Archiviazione automatica come venduto/chiuso per annunci non venduti da oltre 1
anno'
DO
BEGIN
    UPDATE `Annuncio`
    SET `Venduto` = 1
    WHERE `Venduto` = 0
    AND `DataPubblicazione` < (CURRENT_TIMESTAMP - INTERVAL 1 YEAR);
END$$

DELIMITER ;
```

Viste

Le due viste definite per l'architettura della base di dati sono:

1) **annunci_venduti**

Fornisce l'elenco degli annunci contrassegnati come venduti, esponendo i dati identificativi dell'annuncio, il prezzo finale, l'autore e la categoria di appartenenza. Questa vista è impiegata da `genera_report_vendite` (di cui ne parleremo dopo) per calcolare il numeratore della percentuale di successo delle vendite.

```
CREATE
ALGORITHM=UNDEFINED
DEFINER=`root`@`localhost`
SQL SECURITY DEFINER
VIEW `annunci_venduti` AS
select
`annuncio`.`IdAnnuncio` AS
`IdAnnuncio`,`annuncio`.`Titolo` AS
`Titolo`,`annuncio`.`Descrizione` AS
```

```

`Descrizione`,`annuncio`.`Prezzo` AS
`Prezzo`,`annuncio`.`DataPubblicazione` AS
`DataPubblicazione`,`annuncio`.`Autore` AS
`Autore`,`annuncio`.`Categoria` AS
`Categoria`
from `annuncio`
where (`annuncio`.`Venduto` = 1)

```

2) **bacheca_pubblica**

Rappresenta la bacheca operativa accessibile da chiunque per la consultazione degli annunci attivi. Mostra soltanto le inserzioni ancora acquistabili, escludendo automaticamente quelle chiuse o vendute. Viene interrogata direttamente da `visualizza_annunci_non_venduti` di cui andremo a parlare nel paragrafo successivo.

```

CREATE
ALGORITHM=UNDEFINED
DEFINER=`root`@`localhost`
SQL SECURITY DEFINER
VIEW `bacheca_pubblica` AS
select
`a`.`IdAnnuncio` AS
`IdAnnuncio`,`a`.`Titolo` AS
`Titolo`,`a`.`Descrizione` AS
`Descrizione`,`a`.`Prezzo` AS
`Prezzo`,`a`.`DataPubblicazione` AS
`DataPubblicazione`,`a`.`Autore` AS
`Autore`,`c`.`NomeCategoria` AS
`Categoria`
from (`annuncio` `a` join `categoria` `c` on((`a`.`Categoria` = `c`.`IdCategoria`)))
where (`a`.`Venduto` = 0)

```

Stored Procedures e transazioni

1) **annuncio_venduto**

Questa procedura imposta lo stato di un annuncio come venduto dopo aver accertato la titolarità dell'autore e che l'inserzione non risulti già conclusa. Come livello di isolamento ho utilizzato `READ COMMITTED` per prevenire aggiornamenti conflittuali basandosi

unicamente sullo stato finale registrato sul database, innescando a catena la pulizia automatica dei monitoraggi tramite trigger.

```
CREATE DEFINER=`root`@`localhost` PROCEDURE `annuncio_venduto`(  
    IN p_id_annuncio INT,  
    IN p_autore VARCHAR(50)  
)  
BEGIN  
    DECLARE v_autore_effettivo VARCHAR(50);  
    DECLARE v_venduto TINYINT(1);  
  
    DECLARE EXIT HANDLER FOR SQLEXCEPTION  
    BEGIN  
        ROLLBACK;  
        RESIGNAL;  
    END;  
  
    SET TRANSACTION ISOLATION LEVEL READ COMMITTED;  
    START TRANSACTION;  
  
    SELECT `Autore`, `Venduto`  
    INTO v_autore_effettivo, v_venduto  
    FROM `Annuncio`  
    WHERE `IdAnnuncio` = p_id_annuncio;  
  
    IF v_autore_effettivo IS NULL THEN  
        SIGNAL SQLSTATE '45000'  
        SET MESSAGE_TEXT = 'Annuncio inesistente.';  
    END IF;  
  
    IF v_autore_effettivo <> p_autore THEN  
        SIGNAL SQLSTATE '45000'  
        SET MESSAGE_TEXT = 'Operazione non consentita: solo l'autore può contrassegnare l'  
annuncio come venduto.';  
    END IF;
```

```
IF v_venduto = 1 THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'L annuncio risulta già contrassegnato come venduto.';
END IF;
```

```
UPDATE `Annuncio`
SET `Venduto` = 1
WHERE `IdAnnuncio` = p_id_annuncio;
```

```
COMMIT;
```

```
END ;;
```

2) crea_categoria

Questa procedura convalida l'autorizzazione dell'amministratore, l'univocità del nome e l'assenza di cicli diretti o indiretti risalendo la gerarchia dei padri prima di registrare atomicamente la nuova categoria nel database. Come livello di isolamento ho utilizzato READ COMMITTED che è stato scelto per evitare la lettura di transazioni non confermate durante la risalita dell'albero dei padri, garantendo al contempo prestazioni ottimali e riducendo al minimo i lock rispetto a livelli più restrittivi.

```
CREATE DEFINER=`root`@`localhost` PROCEDURE `crea_categoria`(
    IN p_nome_categoria VARCHAR(50),
    IN p_padre INT,
    IN p_amministratore VARCHAR(50),
    OUT p_id_categoria INT
)
BEGIN
    DECLARE v_cursore_padre INT;
    DECLARE v_nome_padre VARCHAR(50);

    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        RESIGNAL;
    END;
```

```
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
START TRANSACTION;

IF NOT EXISTS (SELECT 1 FROM `Amministratore` WHERE `Username` =
p_amministratore) THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Operazione riservata esclusivamente agli amministratori del
sistema.';
END IF;

IF EXISTS (SELECT 1 FROM `Categoria` WHERE LOWER(`NomeCategoria`) =
LOWER(p_nome_categoria)) THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Nome categoria gia presente a sistema.';
END IF;

IF p_padre IS NOT NULL THEN
    SELECT `NomeCategoria` INTO v_nome_padre
    FROM `Categoria`
    WHERE `IdCategoria` = p_padre;

    IF v_nome_padre IS NULL THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'La categoria genitore specificata non esiste.';
    END IF;

    IF LOWER(v_nome_padre) = LOWER(p_nome_categoria) THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Una categoria non puo essere definita come sottocategoria
di se stessa.';
    END IF;

    SET v_cursore_padre = p_padre;
```

```

WHILE v_cursore_padre IS NOT NULL DO
    SELECT `Padre`, `NomeCategoria` INTO v_cursore_padre, v_nome_padre
    FROM `Categoria`
    WHERE `IdCategoria` = v_cursore_padre;

    IF v_nome_padre IS NOT NULL AND LOWER(v_nome_padre) =
    LOWER(p_nome_categoria) THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Rilevato ciclo nella gerarchia dei padri: una categoria
non puo essere sottocategoria di se stessa.';
    END IF;
END WHILE;
END IF;

INSERT INTO `Categoria` (`NomeCategoria`, `Padre`, `Amministratore`)
VALUES (p_nome_categoria, p_padre, p_amministratore);

SET p_id_categoria = LAST_INSERT_ID();

COMMIT;
END ;;

```

3) **genera_report_vendite**

Questa procedura calcola in forma aggregata per ciascun utente il totale degli annunci inseriti, quelli venduti e la corrispondente percentuale di successo delle vendite. Come livello di isolamento ho utilizzato REPEATABLE READ per mantenere coerenti i dati del report, evitando discrepanze dovute a nuovi annunci o vendite durante l'esecuzione.

```

CREATE DEFINER=`root`@`localhost` PROCEDURE `genera_report_vendite`(
    IN p_amministratore VARCHAR(50)
)
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        RESIGNAL;
    END;

```

```
END;
```

```
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
```

```
START TRANSACTION;
```

```
IF NOT EXISTS (SELECT 1 FROM `Amministratore` WHERE `Username` =  
p_amministratore) THEN
```

```
    SIGNAL SQLSTATE '45000'
```

```
    SET MESSAGE_TEXT = 'Operazione consentita solo agli amministratori.';
```

```
END IF;
```

```
SELECT
```

```
    u.`Username`,
```

```
    u.`Nome`,
```

```
    u.`Cognome`,
```

```
    COUNT(a.`IdAnnuncio`) AS `TotaleAnnunci`,
```

```
    COUNT(av.`IdAnnuncio`) AS `AnnunciVenduti`,
```

```
    ROUND(
```

```
        CASE
```

```
            WHEN COUNT(a.`IdAnnuncio`) = 0 THEN 0.00
```

```
            ELSE (COUNT(av.`IdAnnuncio`) * 100.0 / COUNT(a.`IdAnnuncio`))
```

```
        END, 2
```

```
    ) AS `PercentualeSuccessoVendita`
```

```
FROM `Utente_Standard` u
```

```
LEFT JOIN `Annuncio` a ON u.`Username` = a.`Autore`
```

```
LEFT JOIN `annunci_venduti` av ON a.`IdAnnuncio` = av.`IdAnnuncio`
```

```
GROUP BY u.`Username`, u.`Nome`, u.`Cognome`;
```

```
COMMIT;
```

```
END ;;
```

4) **invio_commento**

Questa procedura inserisce un nuovo commento pubblico su un annuncio attivo dopo aver validato il testo e l'identità del commentatore. Come livello di isolamento ho utilizzato READ

COMMITTED perché offre prestazioni ottimali e concorrenza elevata per frequenti inserimenti di commenti verificando in tempo reale l'esistenza delle entità correlate.

```
CREATE DEFINER=`root`@`localhost` PROCEDURE `invio_commento`(  
    IN p_commentatore VARCHAR(50),  
    IN p_id_annuncio INT,  
    IN p_testo TEXT  
)  
BEGIN  
    DECLARE EXIT HANDLER FOR SQLEXCEPTION  
    BEGIN  
        ROLLBACK;  
        RESIGNAL;  
    END;  
  
    SET TRANSACTION ISOLATION LEVEL READ COMMITTED;  
    START TRANSACTION;  
  
    IF TRIM(IFNULL(p_testo, '')) = '' THEN  
        SIGNAL SQLSTATE '45000'  
        SET MESSAGE_TEXT = 'Il testo del commento non puo essere vuoto.';  
    END IF;  
  
    IF NOT EXISTS (SELECT 1 FROM `Utente_Standard` WHERE `Username` =  
p_commentatore) THEN  
        SIGNAL SQLSTATE '45000'  
        SET MESSAGE_TEXT = 'Utente commentatore non valido o inesistente.';  
    END IF;  
  
    INSERT INTO `Commento` (`Commentatore`, `DataOra`, `Testo`, `Annuncio`)  
    VALUES (p_commentatore, CURRENT_TIMESTAMP, p_testo, p_id_annuncio);  
  
    COMMIT;  
END ;;
```

5) **invio_messaggio**

Questa procedura registra lo scambio di una comunicazione privata tra acquirente e venditore riferita a uno specifico annuncio attivo. Come livello di isolamento ho utilizzato READ COMMITTED che permette di inserire i messaggi in tempo reale garantendo che mittente e destinatario esistano sui dati confermati.

```
CREATE DEFINER=`root`@`localhost` PROCEDURE `invio_messaggio`(  
    IN p_mittente VARCHAR(50),  
    IN p_destinatario VARCHAR(50),  
    IN p_id_annuncio INT,  
    IN p_testo TEXT  
)  
BEGIN  
    DECLARE EXIT HANDLER FOR SQLEXCEPTION  
    BEGIN  
        ROLLBACK;  
        RESIGNAL;  
    END;  
  
    SET TRANSACTION ISOLATION LEVEL READ COMMITTED;  
    START TRANSACTION;  
  
    IF TRIM(IFNULL(p_testo, '')) = '' THEN  
        SIGNAL SQLSTATE '45000'  
        SET MESSAGE_TEXT = 'Il testo del messaggio non puo essere vuoto.';  
    END IF;  
  
    IF NOT EXISTS (SELECT 1 FROM `Utente_Standard` WHERE `Username` =  
p_mittente) THEN  
        SIGNAL SQLSTATE '45000'  
        SET MESSAGE_TEXT = 'Mittente non valido o inesistente.';  
    END IF;  
  
    IF NOT EXISTS (SELECT 1 FROM `Utente_Standard` WHERE `Username` =  
p_destinatario) THEN  
        SIGNAL SQLSTATE '45000'
```

```
SET MESSAGE_TEXT = 'Destinatario non valido o inesistente.';
END IF;
```

```
INSERT INTO `Messaggio` (`Mittente`, `Destinatario`, `DataOra`, `Testo`, `Annuncio`)
VALUES (p_mittente, p_destinatario, CURRENT_TIMESTAMP, p_testo,
p_id_annuncio);
```

```
COMMIT;
END ;;
```

6) login

Questa procedura verifica la corrispondenza delle credenziali fornite e restituisce il ruolo associato all'utente. Come livello di isolamento ho utilizzato READ COMMITTED che permette di validare l'identità dell'utente sullo stato reale e confermato delle credenziali, evitando letture sporche derivanti da registrazioni concorrenti in corso di rollback.

```
CREATE DEFINER=`root`@`localhost` PROCEDURE `login`(
    IN p_username VARCHAR(50),
    IN p_password VARCHAR(255),
    OUT p_ruolo ENUM('standard', 'amministratore')
)
```

```
BEGIN
```

```
    DECLARE v_password_hash VARCHAR(255);
    DECLARE v_ruolo ENUM('standard', 'amministratore');
```

```
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
```

```
    BEGIN
```

```
        ROLLBACK;
```

```
        RESIGNAL;
```

```
    END;
```

```
    SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
```

```
    START TRANSACTION;
```

```
    SELECT `Password`, `Ruolo`
    INTO v_password_hash, v_ruolo
```

```
FROM `Credenziali`
```

```
WHERE `Username` = p_username;
```

```
IF v_password_hash IS NULL OR v_password_hash <> SHA2(p_password, 256) THEN
```

```
    SIGNAL SQLSTATE '45000'
```

```
    SET MESSAGE_TEXT = 'Credenziali non valide.';
```

```
END IF;
```

```
SET p_ruolo = v_ruolo;
```

```
COMMIT;
```

```
END ;;
```

7) **modifica_annuncio**

Questa procedura aggiorna il titolo, la descrizione e il prezzo di un annuncio attivo solo se l'utente richiedente coincide con l'autore effettivo del bene. Come livello di isolamento ho usato READ COMMITTED che legge lo stato committed dell'annuncio per certificare che non sia stato già venduto prima di applicare la modifica.

```
CREATE DEFINER=`root`@`localhost` PROCEDURE `modifica_annuncio`(  
    IN p_id_annuncio INT,  
    IN p_autore VARCHAR(50),  
    IN p_nuovo_titolo VARCHAR(100),  
    IN p_nuova_descrizione TEXT,  
    IN p_nuovo_prezzo DECIMAL(10,2)  
)  
BEGIN  
    DECLARE v_autore_effettivo VARCHAR(50);  
    DECLARE v_venduto TINYINT(1);  
  
    DECLARE EXIT HANDLER FOR SQLEXCEPTION  
    BEGIN  
        ROLLBACK;  
        RESIGNAL;  
    END;  
END;
```

```
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;  
START TRANSACTION;
```

```
IF p_nuovo_prezzo < 0 THEN  
    SIGNAL SQLSTATE '45000'  
    SET MESSAGE_TEXT = 'Il prezzo dell annuncio non puo essere negativo.';  
END IF;
```

```
SELECT `Autore`, `Venduto`  
INTO v_autore_effettivo, v_venduto  
FROM `Annuncio`  
WHERE `IdAnnuncio` = p_id_annuncio;
```

```
IF v_autore_effettivo IS NULL THEN  
    SIGNAL SQLSTATE '45000'  
    SET MESSAGE_TEXT = 'Annuncio inesistente.';  
END IF;
```

```
IF v_autore_effettivo <> p_autore THEN  
    SIGNAL SQLSTATE '45000'  
    SET MESSAGE_TEXT = 'Operazione non consentita: solo l autore puo modificare il  
proprio annuncio.';  
END IF;
```

```
IF v_venduto = 1 THEN  
    SIGNAL SQLSTATE '45000'  
    SET MESSAGE_TEXT = 'Impossibile modificare un annuncio contrassegnato come  
venduto.';  
END IF;
```

```
UPDATE `Annuncio`  
SET `Titolo` = p_nuovo_titolo,  
    `Descrizione` = p_nuova_descrizione,  
    `Prezzo` = p_nuovo_prezzo
```

```
WHERE `IdAnnuncio` = p_id_annuncio;
```

```
COMMIT;
```

```
END ;;
```

8) **monitora_annuncio**

Questa procedura associa un utente standard a un annuncio per abilitare le notifiche di modifica, previa verifica che l'annuncio sia invenduto, non monitorato in precedenza e non di proprietà dell'utente stesso. Come livello di isolamento ho utilizzato READ COMMITTED che consente di valutare i controlli di integrità sullo stato corrente dell'annuncio ed evitare l'inserimento di monitoraggi su inserzioni già archiviate.

```
CREATE DEFINER=`root`@`localhost` PROCEDURE `monitora_annuncio`(  
    IN p_username VARCHAR(50),  
    IN p_id_annuncio INT  
)  
BEGIN  
    DECLARE v_venduto TINYINT(1);  
    DECLARE v_autore VARCHAR(50);  
  
    DECLARE EXIT HANDLER FOR SQLEXCEPTION  
    BEGIN  
        ROLLBACK;  
        RESIGNAL;  
    END;  
  
    SET TRANSACTION ISOLATION LEVEL READ COMMITTED;  
    START TRANSACTION;  
  
    IF NOT EXISTS (SELECT 1 FROM `Utente_Standard` WHERE `Username` =  
p_username) THEN  
        SIGNAL SQLSTATE '45000'  
        SET MESSAGE_TEXT = 'Utente specificato non valido o inesistente.';  
    END IF;  
  
    SELECT `Venduto`, `Autore` INTO v_venduto, v_autore
```

```
FROM `Annuncio`
```

```
WHERE `IdAnnuncio` = p_id_annuncio;
```

```
IF v_venduto IS NULL THEN
```

```
    SIGNAL SQLSTATE '45000'
```

```
    SET MESSAGE_TEXT = 'Annuncio inesistente.';
```

```
END IF;
```

```
IF v_venduto = 1 THEN
```

```
    SIGNAL SQLSTATE '45000'
```

```
    SET MESSAGE_TEXT = 'Impossibile monitorare un annuncio già venduto.';
```

```
END IF;
```

```
IF v_autore = p_username THEN
```

```
    SIGNAL SQLSTATE '45000'
```

```
    SET MESSAGE_TEXT = 'Non è consentito monitorare i propri annunci.';
```

```
END IF;
```

```
IF EXISTS (SELECT 1 FROM `Monitoraggio` WHERE `Username` = p_username AND  
`IdAnnuncio` = p_id_annuncio) THEN
```

```
    SIGNAL SQLSTATE '45000'
```

```
    SET MESSAGE_TEXT = 'L annuncio risulta già monitorato da questo utente.';
```

```
END IF;
```

```
INSERT INTO `Monitoraggio` (`Username`, `IdAnnuncio`, `Notificato`)
```

```
VALUES (p_username, p_id_annuncio, 0);
```

```
COMMIT;
```

```
END ;;
```

9) **pubblicazione_annuncio**

Questa procedura crea un nuovo annuncio associato a una categoria e a un autore validi, restituendo tramite parametro di output il codice identificativo generato. Come livello di isolamento ho utilizzato READ COMMITTED per verificare autore e categoria su dati confermati, evitando annunci collegati a entità non valide.

```
CREATE DEFINER=`root`@`localhost` PROCEDURE `pubblicazione_annuncio`(  
    IN p_titolo VARCHAR(100),  
    IN p_descrizione TEXT,  
    IN p_prezzo DECIMAL(10,2),  
    IN p_autore VARCHAR(50),  
    IN p_categoria INT,  
    OUT p_id_annuncio INT  
)  
BEGIN  
    DECLARE EXIT HANDLER FOR SQLEXCEPTION  
    BEGIN  
        ROLLBACK;  
        RESIGNAL;  
    END;  
  
    SET TRANSACTION ISOLATION LEVEL READ COMMITTED;  
    START TRANSACTION;  
  
    IF p_prezzo < 0 THEN  
        SIGNAL SQLSTATE '45000'  
        SET MESSAGE_TEXT = 'Il prezzo dell annuncio non puo essere negativo.';  
    END IF;  
  
    IF NOT EXISTS (SELECT 1 FROM `Utente_Standard` WHERE `Username` = p_autore)  
    THEN  
        SIGNAL SQLSTATE '45000'  
        SET MESSAGE_TEXT = 'Autore specificato non valido o inesistente.';  
    END IF;  
  
    IF NOT EXISTS (SELECT 1 FROM `Categoria` WHERE `IdCategoria` = p_categoria)  
    THEN  
        SIGNAL SQLSTATE '45000'  
        SET MESSAGE_TEXT = 'Categoria specificata inesistente.';  
    END IF;
```

```
INSERT INTO `Annuncio` (`Titolo`, `Descrizione`, `Prezzo`, `Autore`, `Categoria`,  
`Venduto`)  
VALUES (p_titolo, p_descrizione, p_prezzo, p_autore, p_categoria, 0);  
  
SET p_id_annuncio = LAST_INSERT_ID();  
  
COMMIT;  
END ;;
```

10) **registrazione_amministratore**

Questa procedura registra un nuovo utente con privilegi gestionali popolando in un'unica transazione la tabella delle credenziali e quella anagrafica degli amministratori. Come livello di isolamento ho utilizzato READ COMMITTED che permette di validare l'univocità dello username sui dati confermati evitando letture sporche di altre registrazioni contemporanee.

```
CREATE DEFINER=`root`@`localhost` PROCEDURE `registrazione_amministratore`(  
    IN p_username VARCHAR(50),  
    IN p_password VARCHAR(255),  
    IN p_nome VARCHAR(50),  
    IN p_cognome VARCHAR(50),  
    IN p_data_nascita DATE  
)  
BEGIN  
    DECLARE EXIT HANDLER FOR SQLEXCEPTION  
    BEGIN  
        ROLLBACK;  
        RESIGNAL;  
    END;  
  
    SET TRANSACTION ISOLATION LEVEL READ COMMITTED;  
    START TRANSACTION;  
  
    IF EXISTS (SELECT 1 FROM `Credenziali` WHERE `Username` = p_username) THEN  
        SIGNAL SQLSTATE '45000'  
        SET MESSAGE_TEXT = 'Username gia in uso.';
```

```
END IF;
```

```
INSERT INTO `Credenziali` (`Username`, `Password`, `Ruolo`)  
VALUES (p_username, SHA2(p_password, 256), 'amministratore');
```

```
INSERT INTO `Amministratore` (`Username`, `Nome`, `Cognome`, `DataNascita`)  
VALUES (p_username, p_nome, p_cognome, p_data_nascita);
```

```
COMMIT;
```

```
END ;;
```

11) **registrazione_contatto**

Questa procedura aggiunge un nuovo recapito per l'utente, gestendo l'eventuale revoca del flag del contatto preferito precedente se il nuovo recapito è contrassegnato come preferito. Come livello di isolamento ho utilizzato READ COMMITTED che fornisce la coerenza necessaria durante la sequenza di update e insert sui recapiti senza bloccare inutilmente le letture parallele di altri utenti.

```
CREATE DEFINER=`root`@`localhost` PROCEDURE `registrazione_contatto`(  
    IN p_username VARCHAR(50),  
    IN p_tipo ENUM('email', 'telefono', 'cellulare'),  
    IN p_valore VARCHAR(100),  
    IN p_preferito TINYINT(1)  
)  
BEGIN  
    DECLARE EXIT HANDLER FOR SQLEXCEPTION  
    BEGIN  
        ROLLBACK;  
        RESIGNAL;  
    END;  
  
    SET TRANSACTION ISOLATION LEVEL READ COMMITTED;  
    START TRANSACTION;  
  
    IF NOT EXISTS (SELECT 1 FROM `Utente_Standard` WHERE `Username` =  
p_username) THEN
```

```
SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT = 'Utente standard non valido o inesistente.';
END IF;

IF EXISTS (
    SELECT 1 FROM `Contatto`
    WHERE `Username` = p_username AND `Tipo` = p_tipo AND `Valore` = p_valore
) THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Recapito di contatto gia presente per questo utente.';
END IF;

IF p_preferito = 1 THEN
    UPDATE `Contatto`
    SET `Preferito` = 0
    WHERE `Username` = p_username AND `Preferito` = 1;
END IF;

INSERT INTO `Contatto` (`Username`, `Tipo`, `Valore`, `Preferito`)
VALUES (p_username, p_tipo, p_valore, IFNULL(p_preferito, 0));

COMMIT;
END ;;
```

12) registrazione_utente_standard

Questa procedura registra atomicamente un nuovo utente standard inserendo le credenziali di accesso, le informazioni anagrafiche e il primo contatto preferito. Come livello di isolamento ho utilizzato READ COMMITTED che garantisce l'accesso allo stato effettivo e consolidato delle tabelle per verificare che l'username non sia già occupato, massimizzando la concorrenza con scritture immediate.

```
CREATE DEFINER=`root`@`localhost` PROCEDURE `registrazione_utente_standard`(
    IN p_username VARCHAR(50),
    IN p_password VARCHAR(255),
    IN p_nome VARCHAR(50),
    IN p_cognome VARCHAR(50),
```

```
IN p_data_nascita DATE,
IN p_ind_residenza VARCHAR(100),
IN p_ind_fatturazione VARCHAR(100),
IN p_contatto_valore VARCHAR(100),
IN p_contatto_tipo ENUM('email', 'telefono', 'cellulare')
)
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        RESIGNAL;
    END;

    SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
    START TRANSACTION;

    IF EXISTS (SELECT 1 FROM `Credenziali` WHERE `Username` = p_username) THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Username gia in uso.';
    END IF;

    INSERT INTO `Credenziali` (`Username`, `Password`, `Ruolo`)
    VALUES (p_username, SHA2(p_password, 256), 'standard');

    INSERT INTO `Utente_Standard` (`Username`, `Nome`, `Cognome`, `DataNascita`,
    `IndResidenza`, `IndFatturazione`)
    VALUES (p_username, p_nome, p_cognome, p_data_nascita, p_ind_residenza,
    p_ind_fatturazione);

    INSERT INTO `Contatto` (`Username`, `Tipo`, `Valore`, `Preferito`)
    VALUES (p_username, p_contatto_tipo, p_contatto_valore, 1);

    COMMIT;
END ;;
```

13) resetta_flag_notifica

Questa procedura reimposta a zero il valore del campo "Notificato" per uno specifico annuncio monitorato oppure per tutti i monitoraggi dell'utente. Come livello di isolamento ho utilizzato REPEATABLE READ che isola completamente l'operazione di reset garantendo che lo stato delle notifiche aggiornate non sia alterato da transazioni concorrenti durante la fase di elaborazione.

```
CREATE DEFINER='root'@'localhost' PROCEDURE `resetta_flag_notifica`(  
    IN p_username VARCHAR(50),  
    IN p_id_annuncio INT  
)  
BEGIN  
    DECLARE EXIT HANDLER FOR SQLEXCEPTION  
    BEGIN  
        ROLLBACK;  
        RESIGNAL;  
    END;  
  
    SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;  
    START TRANSACTION;  
  
    IF p_id_annuncio IS NOT NULL THEN  
        UPDATE `Monitoraggio`  
        SET `Notificato` = 0  
        WHERE `Username` = p_username AND `IdAnnuncio` = p_id_annuncio;  
    ELSE  
        UPDATE `Monitoraggio`  
        SET `Notificato` = 0  
        WHERE `Username` = p_username;  
    END IF;  
  
    COMMIT;  
END ;;
```

14) verifica_annunci_modificati

Questa procedura esegue un controllo per estrarre gli annunci seguiti dall'utente che hanno registrato modifiche recenti non ancora lette. Come livello di isolamento ho utilizzato READ COMMITTED che rileva tempestivamente gli aggiornamenti di prezzo o descrizione consolidati, operando con la minima interferenza verso altre operazioni di scrittura.

```
CREATE DEFINER=`root`@`localhost` PROCEDURE `verifica_annunci_modificati`(  
    IN p_username VARCHAR(50)  
)  
BEGIN  
    DECLARE EXIT HANDLER FOR SQLEXCEPTION  
    BEGIN  
        ROLLBACK;  
        RESIGNAL;  
    END;  
  
    SET TRANSACTION ISOLATION LEVEL READ COMMITTED;  
    START TRANSACTION;  
  
    SELECT  
        a.`IdAnnuncio`,  
        a.`Titolo`,  
        a.`Prezzo`,  
        a.`Autore`  
    FROM `Monitoraggio` m  
    JOIN `Annuncio` a ON m.`IdAnnuncio` = a.`IdAnnuncio`  
    WHERE m.`Username` = p_username AND m.`Notificato` = 1;  
  
    COMMIT;  
END ;;
```

15) **visualizza_annunci_monitorati**

Questa procedura recupera l'elenco completo degli annunci monitorati da un determinato utente estraendone i dettagli e lo stato di notifica tramite una join tra le tabelle Monitoraggio e Annuncio. Come livello di isolamento ho utilizzato REPEATABLE READ che è stato adottato per garantire una lettura consistente dell'intero set di dati, assicurando che lo stato

degli annunci e dei monitoraggi rimanga stabile e privo di letture non ripetibili durante l'esecuzione della query.

```
CREATE DEFINER=`root`@`localhost` PROCEDURE `visualizza_annunci_monitorati`(  
    IN p_username VARCHAR(50)  
)  
BEGIN  
    DECLARE EXIT HANDLER FOR SQLEXCEPTION  
    BEGIN  
        ROLLBACK;  
        RESIGNAL;  
    END;  
  
    SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;  
    START TRANSACTION;  
  
    SELECT  
        a.`IdAnnuncio`,  
        a.`Titolo`,  
        a.`Descrizione`,  
        a.`Prezzo`,  
        a.`DataPubblicazione`,  
        a.`Autore`,  
        c.`NomeCategoria` AS `Categoria`,  
        m.`Notificato`  
    FROM `Monitoraggio` m  
    JOIN `Annuncio` a ON m.`IdAnnuncio` = a.`IdAnnuncio`  
    JOIN `Categoria` c ON a.`Categoria` = c.`IdCategoria`  
    WHERE m.`Username` = p_username;  
  
    COMMIT;  
END ;;
```

16) **visualizza_annunci_non_venduti**

Questa procedura restituisce la totalità delle inserzioni attive consultabili sulla bacheca pubblica ordinate per data di pubblicazione decrescente. Come livello di isolamento ho

utilizzato READ COMMITTED assicurandoci che gli utenti leggano un catalogo aggiornato alle ultime transazioni consolidate, garantendo elevata concorrenza senza bloccare gli inserimenti simultanei.

```
CREATE DEFINER=`root`@`localhost` PROCEDURE `visualizza_annunci_non_venduti`()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        RESIGNAL;
    END;

    SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
    START TRANSACTION;

    SELECT
        `IdAnnuncio`,
        `Titolo`,
        `Descrizione`,
        `Prezzo`,
        `DataPubblicazione`,
        `Autore`,
        `Categoria`
    FROM `bacheca_pubblica`
    ORDER BY `DataPubblicazione` DESC;

    COMMIT;
END ;;
```

17) **visualizza_annunci_utente**

Questa procedura mostra lo storico di tutti gli annunci (attivi e venduti) pubblicati da uno specifico utente con la relativa categoria. Come livello di isolamento ho utilizzato READ COMMITTED che permette all'utente di consultare istantaneamente il proprio catalogo allineato all'ultimo commit senza subire ritardi di locking.

```
CREATE DEFINER=`root`@`localhost` PROCEDURE `visualizza_annunci_utente`(
    IN p_username VARCHAR(50)
```

```
)  
BEGIN  
    DECLARE EXIT HANDLER FOR SQLEXCEPTION  
    BEGIN  
        ROLLBACK;  
        RESIGNAL;  
    END;  
  
    SET TRANSACTION ISOLATION LEVEL READ COMMITTED;  
    START TRANSACTION;  
  
    SELECT  
        a.`IdAnnuncio`,  
        a.`Titolo`,  
        a.`Descrizione`,  
        a.`Prezzo`,  
        a.`DataPubblicazione`,  
        a.`Venduto`,  
        c.`NomeCategoria` AS `Categoria`  
    FROM `Annuncio` a  
    JOIN `Categoria` c ON a.`Categoria` = c.`IdCategoria`  
    WHERE a.`Autore` = p_username  
    ORDER BY a.`DataPubblicazione` DESC;  
  
    COMMIT;  
END ;;
```

18) **visualizza_commenti_annuncio**

Questa procedura restituisce tutti i commenti lasciati su un determinato annuncio ordinati cronologicamente. Come livello di isolamento ho utilizzato READ COMMITTED che mostra la sequenza di commenti validati e committati fino a quel momento senza trattenere lock di lettura sulle tabelle.

```
CREATE DEFINER=`root`@`localhost` PROCEDURE `visualizza_commenti_annuncio`(  
    IN p_id_annuncio INT  
)
```

```
BEGIN
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
  BEGIN
    ROLLBACK;
    RESIGNAL;
  END;

  SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
  START TRANSACTION;

  SELECT
    `Commentatore`,
    `DataOra`,
    `Testo`
  FROM `Commento`
  WHERE `Annuncio` = p_id_annuncio
  ORDER BY `DataOra` ASC;

  COMMIT;
END ;;
```

19) **visualizza_messaggi_annuncio**

Questa procedura estrae la cronologia dei messaggi privati scambiati per un annuncio a cui partecipa l'utente richiedente. Come livello di isolamento ho utilizzato READ COMMITTED che mostra la conversazione aggiornata all'ultimo stato confermato tutelando la riservatezza e consentendo una lettura immediata dei messaggi ricevuti.

```
CREATE DEFINER=`root`@`localhost` PROCEDURE `visualizza_messaggi_annuncio`(
  IN p_id_annuncio INT,
  IN p_richiedente VARCHAR(50)
)
BEGIN
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
  BEGIN
    ROLLBACK;
    RESIGNAL;
  END;
```

```
END;
```

```
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;  
START TRANSACTION;
```

```
SELECT  
    m.`Mittente`,  
    m.`Destinatario`,  
    m.`DataOra`,  
    m.`Testo`  
FROM `Messaggio` m  
WHERE m.`Annuncio` = p_id_annuncio  
    AND (m.`Mittente` = p_richiedente OR m.`Destinatario` = p_richiedente)  
ORDER BY m.`DataOra` ASC;  
  
COMMIT;  
END ;;
```

20) **visualizza_tutte_categorie**

Questa procedura restituisce l'intero albero gerarchico delle categorie indicando per ciascuna il rispettivo genitore e l'amministratore creatore. Come livello di isolamento ho utilizzato READ COMMITTED che fornisce una visualizzazione immediata della tassonomia consolidata del catalogo, permettendo letture concorrenti veloci da parte di utenti e amministratori.

```
CREATE DEFINER=`root`@`localhost` PROCEDURE `visualizza_tutte_categorie`()  
BEGIN  
    DECLARE EXIT HANDLER FOR SQLEXCEPTION  
    BEGIN  
        ROLLBACK;  
        RESIGNAL;  
    END;  
  
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;  
START TRANSACTION;
```

```
SELECT
    c.`IdCategoria`,
    c.`NomeCategoria`,
    c.`Padre` AS `IdPadre`,
    p.`NomeCategoria` AS `NomePadre`,
    c.`Amministratore` AS `CreatoDa`
FROM `Categoria` c
LEFT JOIN `Categoria` p ON c.`Padre` = p.`IdCategoria`
ORDER BY c.`IdCategoria` ASC;

COMMIT;

END ;;
```

21) imposta_contatto_preferito

Questa procedura imposta un recapito specifico come preferito per l'utente indicato, azzerando contestualmente il flag di preferenza su tutti gli altri suoi contatti registrati. Come livello di isolamento ho utilizzato READ COMMITTED per garantire che l'operazione legga esclusivamente modifiche già confermate evitando le letture sporche, massimizzando la concorrenza del database senza la necessità di blocchi più pesanti per semplici query di verifica e update puntuali.

```
CREATE DEFINER='root'@'localhost' PROCEDURE `imposta_contatto_preferito`(
    IN p_username VARCHAR(50),
    IN p_tipo ENUM('email', 'telefono', 'cellulare'),
    IN p_valore VARCHAR(100)
)
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        RESIGNAL;
    END;

    SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
    START TRANSACTION;
```

```
IF NOT EXISTS (  
    SELECT 1 FROM `Contatto`  
    WHERE `Username` = p_username AND `Tipo` = p_tipo AND `Valore` = p_valore  
) THEN  
    SIGNAL SQLSTATE '45000'  
    SET MESSAGE_TEXT = 'Recapito di contatto specificato non trovato.';  
END IF;
```

```
UPDATE `Contatto`  
SET `Preferito` = 0  
WHERE `Username` = p_username;
```

```
UPDATE `Contatto`  
SET `Preferito` = 1  
WHERE `Username` = p_username AND `Tipo` = p_tipo AND `Valore` = p_valore;
```

```
COMMIT;  
END ;;
```

22) **rimuovi_contatto**

Questa procedura elimina in modo sicuro un determinato recapito associato a un utente, demandando ai trigger di sistema la verifica che non si tratti dell'unico contatto registrato o di quello preferito. Come livello di isolamento ho utilizzato READ COMMITTED per prevenire la lettura di dati non confermati garantendo la coerenza del controllo di esistenza, offrendo al contempo le massime prestazioni di concorrenza senza richiedere blocchi di lettura prolungati per una singola cancellazione mirata.

```
CREATE DEFINER=`root`@`localhost` PROCEDURE `rimuovi_contatto`(  
    IN p_username VARCHAR(50),  
    IN p_tipo ENUM('email', 'telefono', 'cellulare'),  
    IN p_valore VARCHAR(100)  
)  
BEGIN  
    DECLARE EXIT HANDLER FOR SQLEXCEPTION  
    BEGIN  
        ROLLBACK;
```

```
        RESIGNAL;  
    END;  
  
    SET TRANSACTION ISOLATION LEVEL READ COMMITTED;  
    START TRANSACTION;  
  
    IF NOT EXISTS (  
        SELECT 1 FROM `Contatto`  
        WHERE `Username` = p_username AND `Tipo` = p_tipo AND `Valore` = p_valore  
    ) THEN  
        SIGNAL SQLSTATE '45000'  
        SET MESSAGE_TEXT = 'Recapito di contatto non trovato.';  
    END IF;  
  
    DELETE FROM `Contatto`  
    WHERE `Username` = p_username AND `Tipo` = p_tipo AND `Valore` = p_valore;  
  
    COMMIT;  
END ;;
```